

Ch.2 Docker 이해하기

오픈이는 퇴근하고 집으로 돌아와 커피 한 잔을 옆에 두고 노트북을 열었습니다. 오늘 저녁의 목표는 하나였습니다.

Docker가 무엇이고 어떻게 돌아가는지, 개념과 기본 사용법을 손에 익힌다.

선배가 던져준 말 하나에 오늘 저녁을 걸었습니다. 오늘 저녁 안에 다 될 것 같지는 않았지만, 어디까지 갈 수 있는지는 보고 싶었습니다. 설치 버튼에 손이 갔다가 멈췄습니다.

'명령어를 치기 전에 원리부터 보자.'

Docker가 환경을 어떻게 다루는지부터 이해해야 뒤에 나올 명령어들이 기계적으로 외워지지 않을 것 같았습니다.

2.1 가상화 — 가상머신과 컨테이너

2.1.1 주방 네 개 대신 주방 하나

우선 컨테이너가 어떤 원리로 돌아가는지 알아보기로 했습니다. 오픈이는 주방에서 요리사 여러 명이 일하는 장면을 떠올려봤습니다.

한 주방에 요리사 네 명이 있습니다. 네 명이 동시에 요리를 하려면 주방을 어떻게 꾸며야 할까요.



그림 2-1. 한 주방에 네 명의 요리사가 있는 상황

첫 번째 방식은 요리사마다 냉장고, 가스레인지, 조리대를 각자 한 세트씩 사주는 것입니다. 이렇게 하면 네 사람이 완전히 독립된 환경에서 요리하지만, 같은 설비를 네 번 갖춰야 하니 비용이 많이 듭니다.

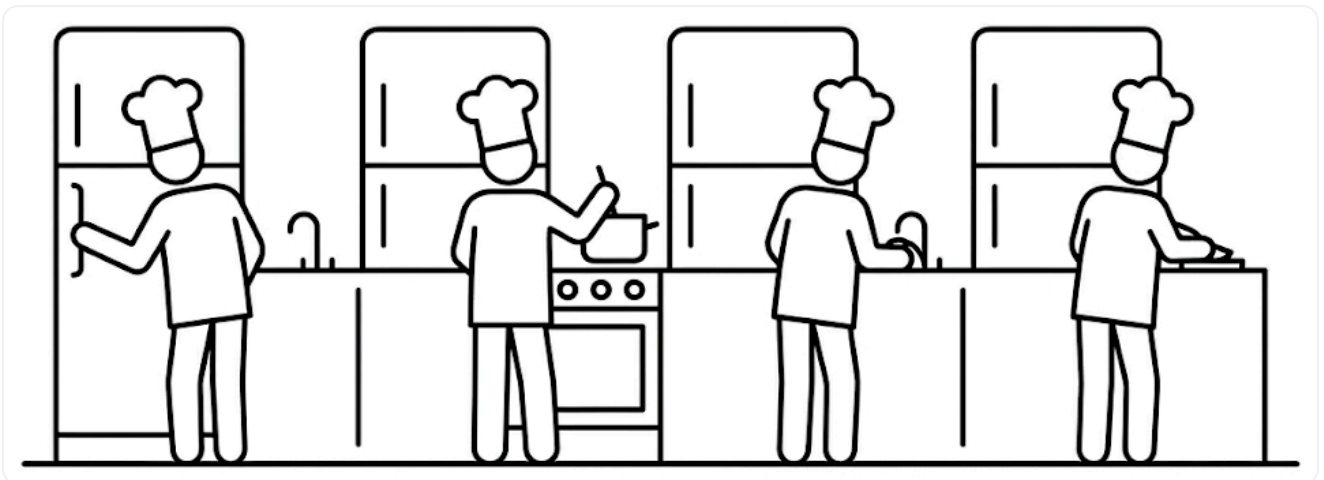


그림 2-2. 주방 설비를 통째로 복제하는 첫 번째 방식

두 번째 방식은 냉장고와 가스레인지 같은 공용 설비는 함께 쓰고, 칼·도마·조리대처럼 개인이 쓸 도구만 각자 챙기는 것입니다. 장비 수는 그대로인데 공간은 네 명이 동시에 쓸 수 있습니다.

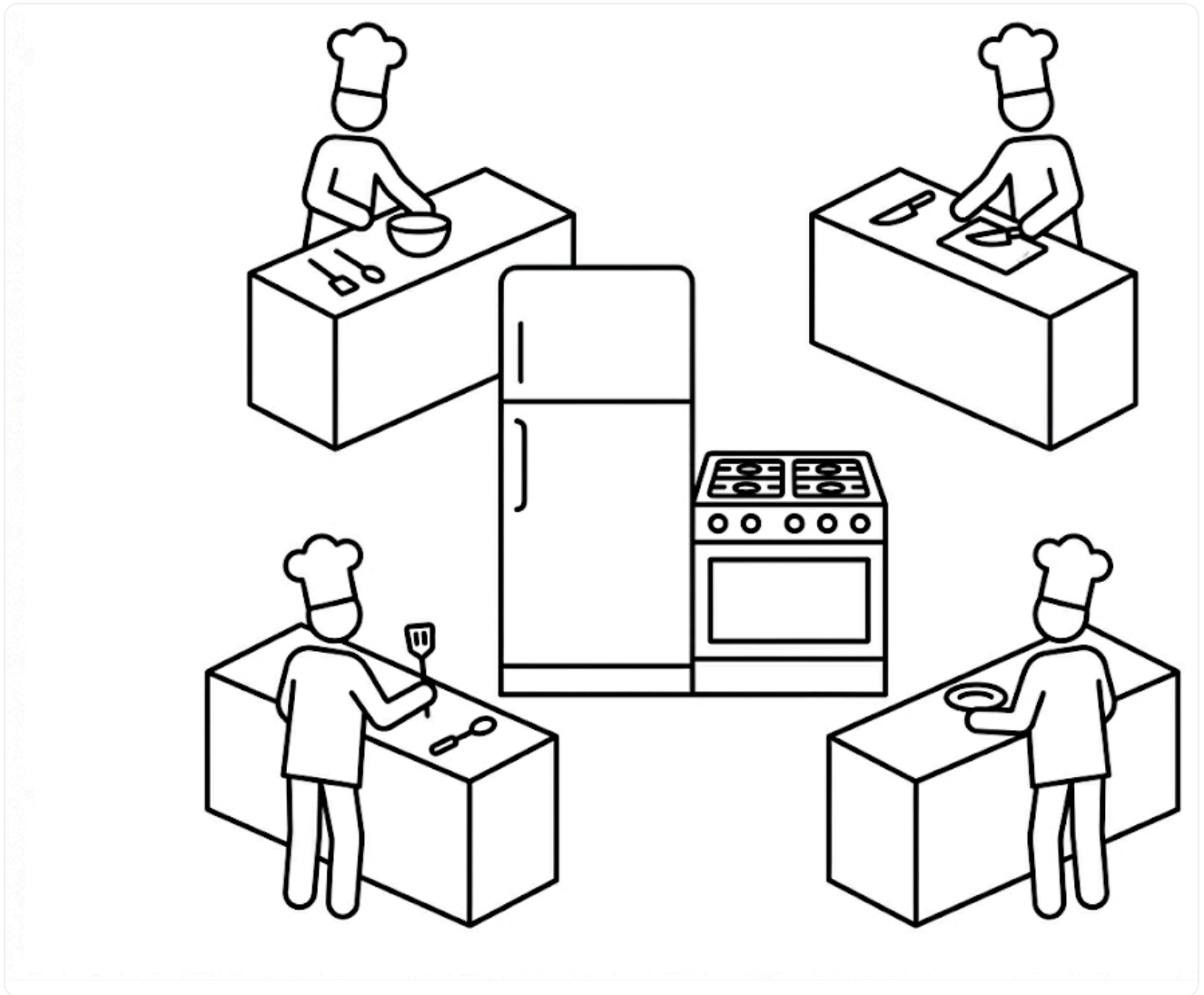


그림 2-3. 공용 설비는 공유하고 개별 공간만 쪼개는 두 번째 방식

이렇게 하나의 주방을 여러 명에서 사용하는 것처럼 하나의 컴퓨터를 여러 대로 나눠서 사용하는 IT 기술을 **가상화(Virtualization)** 라고 부릅니다.

가상화(Virtualization): 하나의 물리 서버를 논리적으로 여러 대처럼 나누어 쓰는 기술입니다. 자원을 효율적으로 쓰면서 서로 다른 환경을 안전하게 격리하는 것이 목적입니다.

가상화에도 두 가지 방식이 있습니다. 설비를 통째로 복제하는 쪽이 **하이퍼바이저 가상화**, 공유하는 쪽이 **컨테이너 가상화**입니다. Docker는 두 번째 방식의 도구입니다. 같은 OS 위에서 여러 앱이 서로 충돌하지 않고 각자 돌게 해 줍니다.

하이퍼바이저 가상화 vs 컨테이너 가상화

하이퍼바이저 가상화(Hypervisor Virtualization) 는 하드웨어 위에 OS 자체를 따로 띄우는 방식이고, 결과물은 **가상 머신(VM)** 입니다. 격리는 완전하지만 OS 전체가 새로 떠야 해서 무겁고 기동이 느립니다. 반면 **컨테이너 가상화(Container Virtualization)** 는 호스트 OS의 커널을 공유하면서 파일시스템·네트워크·프로세스 공간만 따로 두는 방식이고, 결과물은 **컨테이너**입니다. 커널을 공유하므로 가볍고 빠르게 뜹니다.

'아! 도커는 내 컴퓨터 OS 하나를 여러 컨테이너가 나눠 쓰면서, 각자 격리된 공간에서 따로 돌게 하는 거구나.'

2.1.2 IT로 옮긴 구조

비유만 머리에 두면 나중에 실제 용어와 헷갈립니다. 오픈이는 이 주방 그림을 IT 구조로 옮겨 그려봤습니다.

맨 아래에 **하드웨어**, 그 위에 **호스트 OS**, 그 위에 **Docker 엔진**이 있고, Docker 엔진 위에 **컨테이너**가 나란히 떠 있습니다. 각 컨테이너 안에는 애플리케이션(App)과 라이브러리(Library)가 같이 들어갑니다.

주방 비유의 공용 설비와 그걸 운영하는 주방장이 **호스트 OS** , 각자의 조리대가 **컨테이너**입니다.



그림 2-4. 컨테이너 가상화의 전체 구조

그중 주방장이 OS의 핵심, 곧 **커널**입니다. 요리사들은 주방장의 명령에 따라 움직이고, 어떤 위치에서 어떤 역할을 할지도 주방장이 정합니다.

커널(Kernel): 운영체제(OS)의 핵심입니다. 프로세스 생성, 메모리 관리, 격리 공간 생성을 담당합니다. Docker의 모든 컨테이너는 호스트 PC의 **같은 리눅스 커널**을 공유하고, 그 하나의 커널이 각 컨테이너에 격리 공간을 할당합니다.

구조의 핵심은 **호스트 OS의 커널 하나를 모든 컨테이너가 공유**한다는 점입니다. VM처럼 OS를 통째로 따로 올리지 않으니 가볍고 빠르게 뜯니다. Docker 엔진이 그 커널에 "**새 격리 공간을 만들어 달라**"고 요청하면, 실제로 프로세스를 격리하는 일은 **커널**이 합니다. 그래서 다른 컨테이너가 다른 앱과 다른 라이브러리를 가져도 충돌하지 않고 한쪽이 런타임을 업그레이드해도 옆 컨테이너는 영향을 받지 않습니다.

컨테이너의 격리

사실 컨테이너 안에서 도는 것은 처음부터 격리된 무언가가 아니라, 호스트 OS 위에서 다른 것들과 함께 도는 **평범한 프로세스**입니다. 그대로 두면 한 컨테이너의 프로세스가 다른 컨테이너의 파일을 들여다보거나, 두 컨테이너가 같은 포트를 두고 충돌할 수 있습니다. 그래서 **리눅스 커널**은 컨테이너마다 **파일시스템**(`/bin`, `/lib` 같은 폴더), **네트워크**(IP와 포트), **프로세스**(PID 번호) 세 가지 공간을 따로 만들어 줍니다. 이렇게 컨테이너마다 세 공간을 따로 갖는 상태가 바로 **격리**입니다.

2.1.3 컨테이너가 만들어지는 과정

오픈이는 이제 컨테이너의 개념을 잡았지만, **어떻게 만들어지는지**까지는 아직 그림이 그려지지 않았습니다. 컨테이너를 실제로 띄우기 전에 한 줄의 명령 뒤에서 무슨 일이 돌아가는지 눈에 들어와 있어야, 나중에 직접 명령어를 칠 때 기계적으로 외우지 않고 의미를 이해한 채로 칠 수 있을 것 같았습니다.

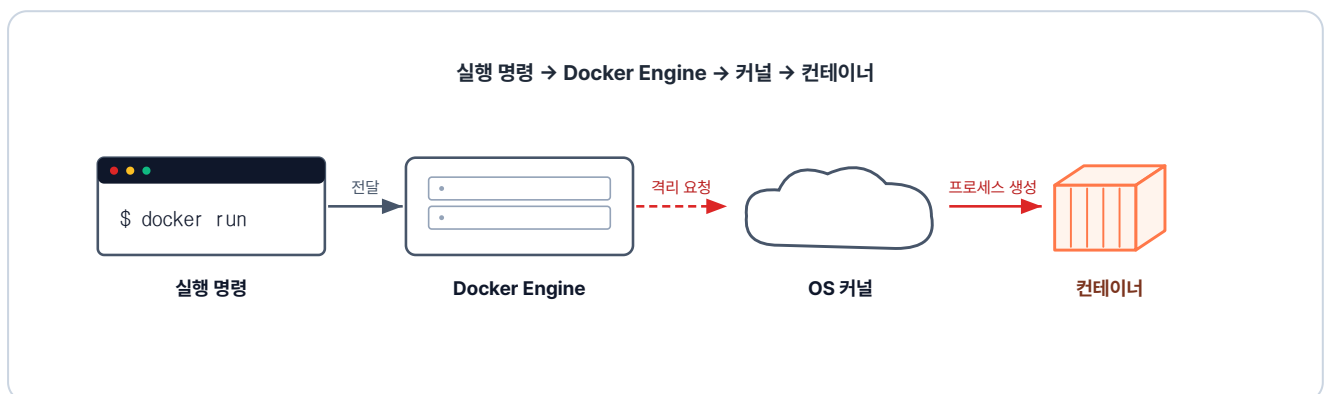


그림 2-5. 컨테이너 실행 명령이 커널의 격리 기능을 타고 컨테이너가 되는 흐름

1. 사용자가 컨테이너를 실행하라는 명령을 친다.

2. **Docker 엔진**이 이 명령을 받고, 로컬에 있는 이미지를 읽는다(없으면 Docker Hub에서 내려받는다).
3. Docker 엔진이 **커널**에게 "**이 이미지 사양으로 격리된 프로세스 하나 만들어 줘**"라고 요청한다.
4. 커널이 격리 공간(파일시스템/네트워크/프로세스)을 만들고, 그 안에서 새 프로세스를 띄운다.
5. 이 격리된 프로세스가 곧 **컨테이너**다.

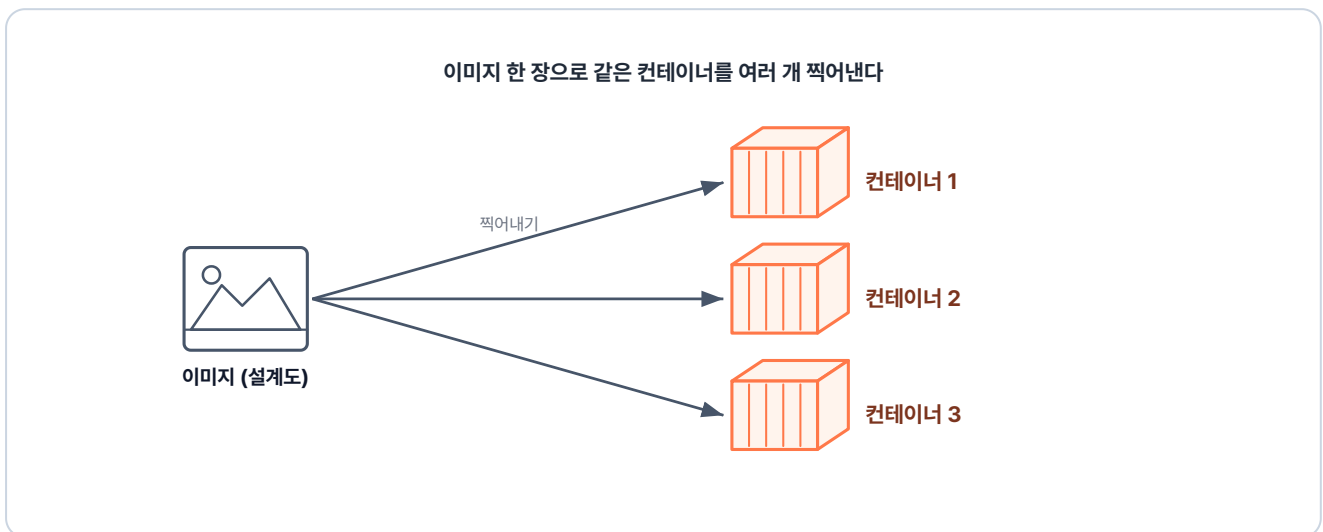
컨테이너를 실제로 만드는 건 Docker 엔진이 아니라 커널입니다. 커널이 주방장이라면, Docker 엔진은 홀의 프런트 직원에 가깝습니다. 새 주문이 들어오면 프런트가 주방장에게 전달합니다. 실제로 새 요리사(프로세스)를 불러 작업을 시작시키는 건 주방장이 합니다.

오픈이는 노트에 "**엔진=프런트, 커널=주방장**"이라고 한 줄 적었습니다. 적고 나니 컨테이너의 정체와 만들어지는 과정이 머릿속에서 정리됐습니다.

'그럼 이 안에서 뭘 실행할지는 어디서 정해지지.'

2.2 이미지 — 컨테이너의 설계도

컨테이너가 격리된 프로세스라는 건 알았지만, 그 프로세스가 무엇을 실행할지를 결정하는 원본이 따로 있어야 합니다. 그 원본이 **이미지**입니다.



이미지는 봉어빵 틀에 가깝습니다. 틀(이미지) 하나로 봉어빵(컨테이너)을 여러 개 찍어낼 수 있고, 틀 자체는 변하지 않습니다. 같은 이미지에서 나온 컨테이너는 어디서 띄우든 똑같은 환경을 가집니다. 이 덕에 "**내 PC와 서버 환경이 같다**"가 가능합니다. 로컬에서 만든 **이미지**를 서버에서 그대로 꺼내 실행하면 서버의 런타임 버전이 뭐든 컨테이너는 자기 이미지 안의 런타임으로 돕니다.

이미지는 **Docker Hub**라는 공용 저장소에서 내려받을 수도 있고, 직접 만들어 올릴 수도 있습니다.

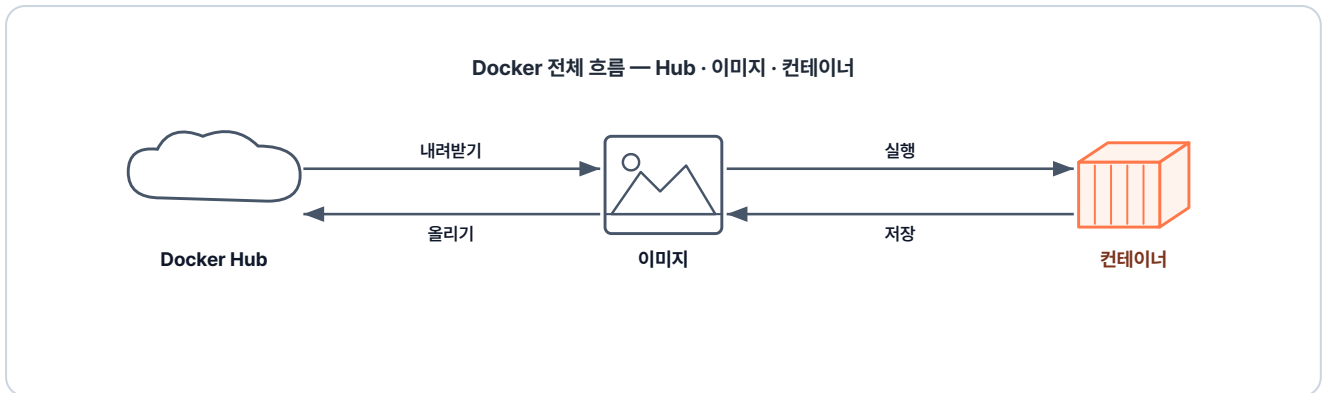


그림 2-7. Docker의 전체 흐름

Docker hub를 활용하면 내 PC에서 세팅한 환경을 서버에서 재현하는 일도, 팀원끼리 환경을 공유하는 일도 가능합니다.

이미지 레이어

이미지는 한 덩어리가 아니라 여러 **레이어**가 층층이 쌓인 구조입니다. 아래쪽에는 OS 기본 라이브러리나 런타임처럼 잘 안 바뀌는 부분이 있고, 위쪽에는 애플리케이션 코드처럼 자주 바뀌는 부분이 있습니다. 한 번 받은 아래 레이어는 저장해 두고 다시 쓰기 때문에, 코드만 바뀐 새 버전이 나오면 맨 위 레이어만 새로 받으면 됩니다. 여기에 커널은 호스트와 공유하므로 이미지에 담지 않습니다. 이런 구조 덕에 도커 이미지는 OS 전체를 담는 VM 이미지에 비해 훨씬 가볍습니다.

원리를 알았으니 이제 Docker를 설치하고 직접 띄워 봅니다.

2.3 Docker 설치 — 실습 환경 준비

Docker Desktop 설치 공식 사이트(<https://www.docker.com/products/docker-desktop/>)에서 OS에 맞는 설치 파일을 받아 기본 옵션대로 설치하면 끝입니다. 오픈이는 설치가 끝난 뒤 터미널에 버전을 찍어봤습니다.

```
docker version    # Docker 버전 확인
```

Client와 Server 정보가 둘 다 뜨면 설치가 완료된 겁니다.

Windows와 macOS의 내부 리눅스

Docker는 리눅스 커널의 기능을 쓰기 때문에 리눅스 기반이 아닌 Windows와 macOS에서는 Docker Desktop이 내부에 리눅스 환경을 한 겹 더 띄웁니다. Windows는 **WSL2**(Windows Subsystem for Linux 2)를, macOS는 경량 Linux VM을 씁니다. 사용자 입장에서는 신경 쓸 일이 없지만 뒤에 나올 네트워크 얘기에서 이 차이가 중요해집니다.

2.4 Docker CLI — 첫 컨테이너 띄우기

2.4.1 docker pull: 이미지 가져오기

가장 먼저 오픈이는 Docker Hub에서 이미지를 내려 받았습니다.

```
docker pull nginx # nginx 이미지 다운로드
```



그림 2-8. nginx 이미지 다운로드 결과

진행 바가 줄줄이 올라가더니 프롬프트가 돌아왔습니다. 이미지가 로컬 저장소에 받아진 것이고, 아직 실행된 건 아닙니다.

2.4.2 docker run: 컨테이너 띄우기

그 다음 다운받은 이미지로 컨테이너를 띄워 봤습니다.


```
docker run nginx # nginx 컨테이너 실행
```

실행 결과

```
$ docker run nginx
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform
configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in
/etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/03/17 10:38:22 [notice] 1#1: using the "epoll" event method
```

그림 2-9. nginx 컨테이너 실행

엔터를 누르자마자 터미널이 뚝 멈춥니다. 커서가 깜빡이지도 않고, 아무 입력도 되지 않았습니다.

'뭐가 잘못된 건가.'

이는 컨테이너가 **포그라운드** 상태로 터미널을 점유하고 있기 때문입니다. 통화 중에는 다른 전화를 받을 수 없는 것과 같은 상황입니다.

우선 오픈이는 터미널에 **CTRL + C** 를 입력해 빠져나왔습니다.

포그라운드(Foreground): 프로세스가 터미널을 독점하는 상태입니다. 프로세스가 끝나거나 강제 종료될 때까지 그 터미널로 다른 명령을 내릴 수 없습니다.

2.4.3 docker run -d: 백그라운드로 띄우기

'컨테이너는 띄운 채로 터미널도 같이 쓰려면 어떻게 해야 하지?'

이 문제를 풀어주는 게 **-d** 옵션입니다. **-d** 옵션을 사용하면 백그라운드 상태에서 프로세스를 실행할 수 있습니다.

```
docker run -d nginx # -d: detached, 백그라운드 실행
```



실행 결과

```
$ docker run -d nginx
4932c46c66589dc651efd5e50a8773d6d077f8a1e005f177385f8e0cc115ad7d
```

그림 2-10. 백그라운드 실행 결과

이번엔 컨테이너 ID가 한 줄 찍히더니 바로 커서가 돌아왔습니다. 방금 멈췄던 터미널과 대비되면서 `-d`의 역할이 분명해졌습니다.

'글자 하나로 포그라운드가 백그라운드가 되네.'

2.4.4 docker ps: 실행 중인 컨테이너 확인

오픈이는 백그라운드에서 돌고 있는 컨테이너가 실제로 살아 있는지 확인했습니다.

```
docker ps # 실행 중인 컨테이너 목록
```



실행 결과

```
$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
4932c46c6658 nginx "/docker-entrypoint...." 1 second ago Up Less than a second dazzling_carso
```

그림 2-11. 컨테이너 목록 조회

목록에 방금 띄운 nginx가 한 줄 찍혔습니다.

2.4.5 자주 쓰는 명령어

명령어	설명
<code>docker pull <이미지명></code>	Docker Hub에서 이미지 다운로드
<code>docker images</code>	로컬에 저장된 이미지 목록
<code>docker logs <컨테이너ID></code>	컨테이너 로그 출력
<code>docker ps -a</code>	종료된 컨테이너 포함 전체 목록
<code>docker stop <컨테이너ID></code>	실행 중인 컨테이너 종료
<code>docker rm <컨테이너ID></code>	종료된 컨테이너 삭제
<code>docker rmi <이미지ID></code>	이미지 삭제

조회는 `ps`, 로그는 `logs`, 내리기는 `stop`, 지우기는 `rm` 을 씁니다. 이 정도로 묶어두면 필요할 때 찾아 쓸 수 있습니다.

여기까지 Docker Hub에서 이미지를 받고 실행을 했습니다. 그다음으로 오픈이의 관심은 컨테이너가 외부와 어떻게 통신하는지로 옮겨갔습니다.

2.5 컨테이너 통신

컨테이너 안에서 앱이 돌아도 외부에서 접속할 수 없으면 쓸모가 없습니다. 격리된 프로세스라는 개념은 익혔지만, 이 격리된 것이 바깥과 어떻게 연결되는지를 모르면 컨테이너 내부를 접근할 수 없습니다. 여기서 컨테이너가 어떻게 통신하는지 큰 그림을 잡아보겠습니다.

2.5.1 네트워크 한눈에 보기

오픈이는 방금 띄운 nginx 컨테이너가 실제로 어떻게 연결되어 있는지 궁금해했습니다. 격리된 공간이라고 했는데 서로 어떻게 오가고, 외부에서는 어떻게 들어가는지.

자료를 훑어보니 컨테이너 네트워크는 세 가지로 정리되어 있습니다. **컨테이너끼리 연결하는 방법, 외부 요청이 컨테이너로 들어오는 길, 이름으로 컨테이너를 찾는 방법**입니다.

오픈이는 이해를 쉽게 하기 위해 **푸드코트**로 그려 봤습니다.

2.5.2 컨테이너끼리 — 주방 모니터와 주문 전광판

A, B, C 식당이 푸드코트에 입점해 있습니다. 식당끼리는 벽으로 완전히 분리되어 있어 서로의 주방을 들여다볼 수 없습니다.

각 식당의 주방에는 **주방 모니터**가 한 대씩 있고, 푸드코트 한가운데에는 모두가 보는 **주문 전광판**이 걸려 있습니다. 주방 모니터와 전광판은 한 가닥의 **전용 통신 케이블**로 이어져 있고, 모든 식당의 케이블이 전광판 본체에 꽂혀 있습니다. 식당끼리 벽이 막혀 있어도 이 전광판으로 서로의 주문 현황을 알 수 있습니다.

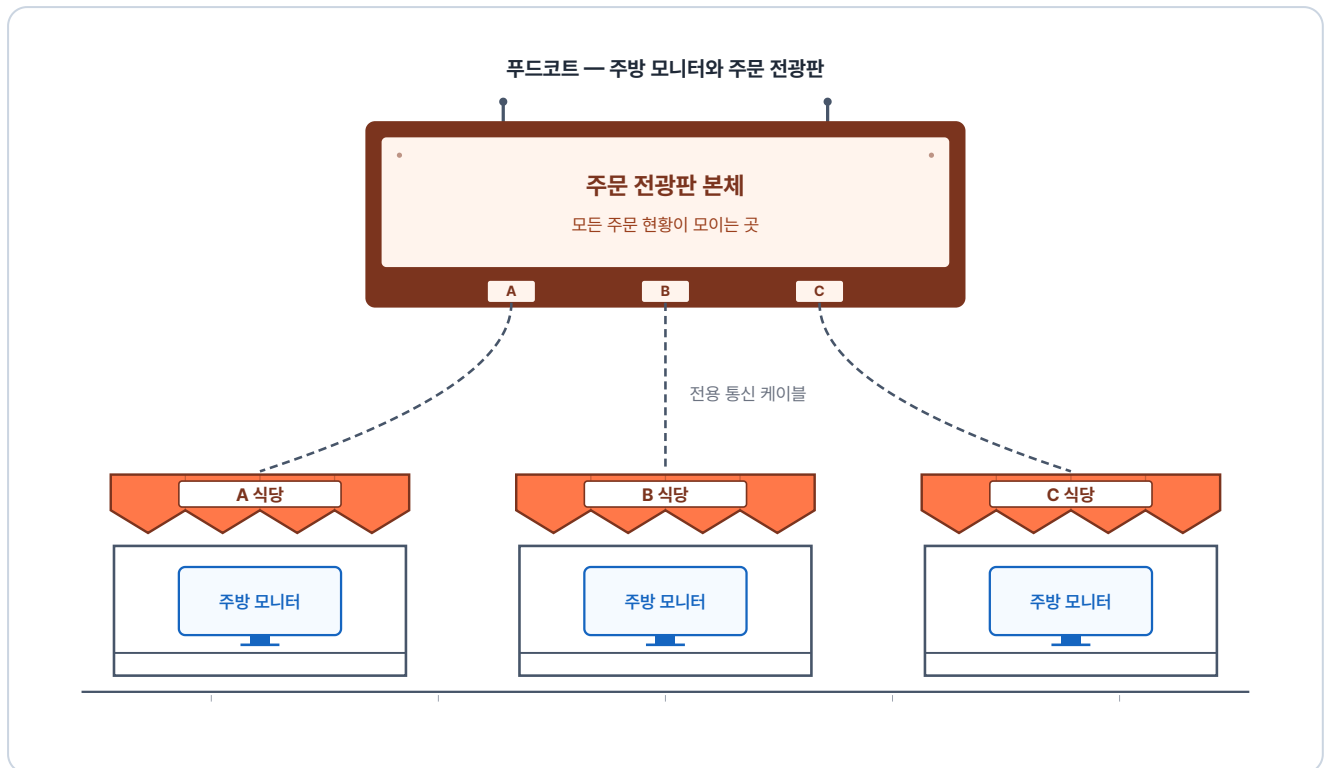


그림 2-12. 주방 모니터·전용 통신 케이블·주문 전광판이 컨테이너 네트워크의 비유

이 비유를 IT로 옮기면 세 가지가 짝지어집니다.

푸드코트	IT 용어	한 줄 설명
벽으로 분리된 주방	Namespace	컨테이너마다 새로 생기는 격리된 네트워크 공간입니다
전용 통신 케이블	veth pair	격리된 두 공간을 잇는 한 쌍짜리 가상 인터페이스입니다
주문 전광판	docker0	같은 브리지에 묶인 컨테이너 사이로 패킷(데이터 묶음)을 중계하는 가상 브리지입니다

컨테이너가 뜨는 순간 자기만의 Namespace가 새로 생기고 veth pair 한 쌍이 만들어집니다. 한 끝은 컨테이너 안에, 다른 끝은 호스트의 docker0에 자동으로 꽂히면서 docker0가 빈 IP(172.17.0.x)를 컨테이너에 할당합니다. 이때 A 컨테이너가 B의 IP로 호출하면 데이터는 A → docker0 → B의 경로로 흐릅니다. docker0는 어떤 케이블이 어느 IP에 묶여 있는지 표를 들고 있어서 들어온 데이터를 정확히 그 IP의 케이블로만 흘려보냅니다.

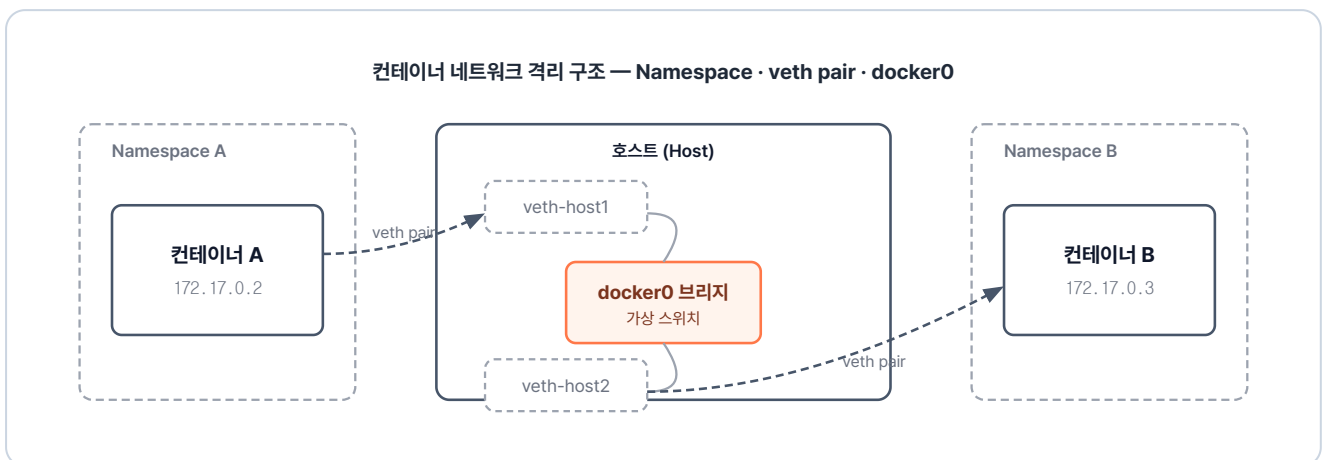


그림 2-13. 컨테이너 네트워크 격리 구조

'각자 주방을 따로 쓰면서 케이블 하나로 전광판 본체에 모이는 거네. 격리와 연결이 한 구조 안에 같이 들어 있구나.'

오픈이가 다음으로 알아본 건 외부 손님이 식당에 어떻게 주문을 넣느냐였습니다.

2.5.3 외부에서 들어오기 — 입구 키오스크

docker0는 도커가 호스트에 기본으로 만들어 두는 **가상 스위치**입니다. 이 위에서 동작하는 네트워크를 **기본 네트워크**라고 부릅니다. 공유기가 집안 기기를 한 네트워크로 묶어주듯, docker0도 같은 호스트의 컨테이너들을 이 기본 네트워크로 묶어둡니다.

그래서 `docker run`으로 컨테이너를 띄우면 별도 옵션을 지정하지 않는 한 자동으로 이 기본 네트워크에 들어갑니다. 같은 네트워크 안에 있으니 IP만 알면 서로 통신할 수 있습니다.

문제는 공유기 너머, 즉 호스트 PC 바깥에 있는 외부 사용자입니다. 브라우저처럼 외부에서 컨테이너로 바로 접근하기는 어렵습니다. 이유는 두 가지입니다.

- Windows와 macOS는 리눅스 기반 운영체제가 아니어서 Docker Desktop이 OS 내부에 **가상의 리눅스 환경**을 만들고 그 안에서 Docker를 실행합니다. docker0와 컨테이너 IP는 모두 이 가상 리눅스 안에 들어 있어서 호스트 (Windows·macOS) 터미널에서는 그 IP로 바로 닿을 수 없습니다.
- 설령 리눅스 환경이라 해도 외부 사용자는 컨테이너의 내부 IP를 알 수 없습니다. 컨테이너가 뜰 때마다 IP가 바뀌기도 하고 바깥으로 공개된 주소도 아니기 때문입니다.

따라서 외부에서 컨테이너로 들어오는 **별도의 통로**가 필요합니다.

푸드코트에 비유하면 외부 손님은 주방에 직접 들어가지 못합니다. 대신 입구의 **키오스크**에 주문을 넣으면 키오스크가 해당 식당으로 그 주문을 넘겨줍니다. 짜장면은 A식당으로, 돈까스는 B식당으로 주문이 전달됩니다.

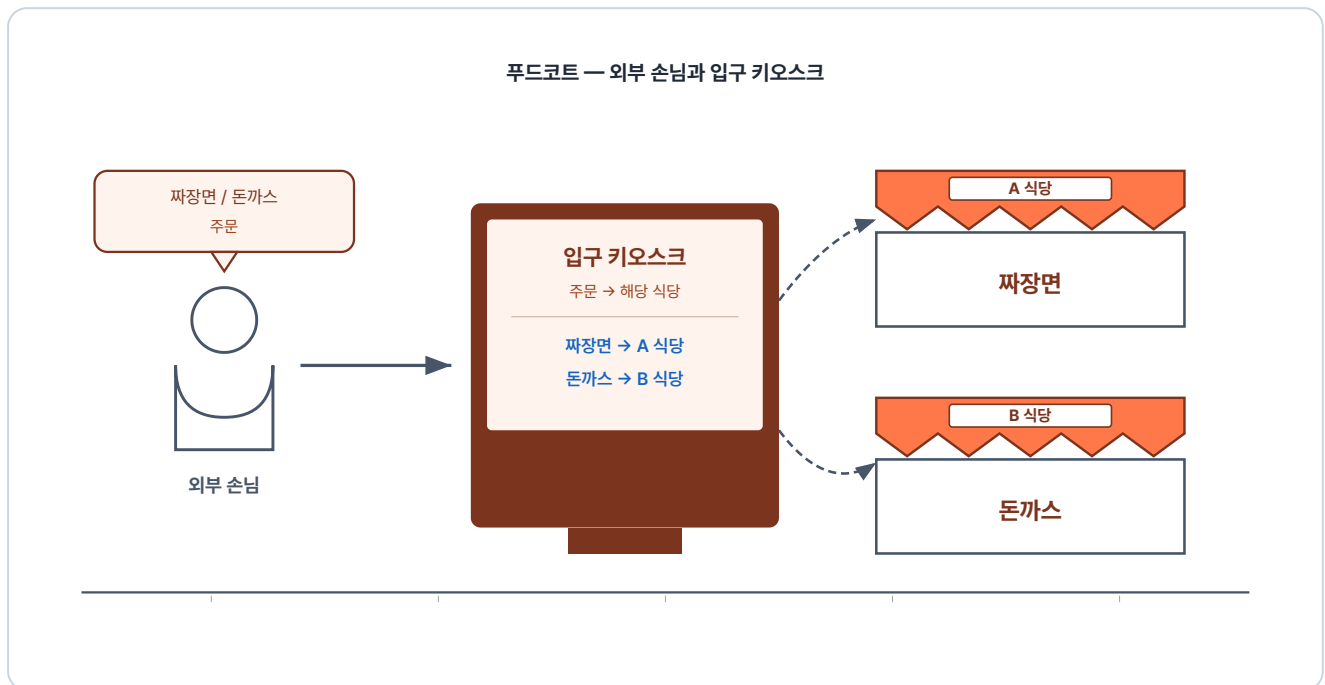


그림 2-14. 입구 키오스크가 손님의 주문을 각 식당으로 넘겨주는 비유

이 비유를 IT로 옮기면 세 가지가 짝지어집니다.

푸드코트	IT 용어	한 줄 설명
입구 키오스크	포트포워딩	호스트 PC의 외부 공개 포트 들어온 요청을 컨테이너 포트 자동으로 넘기는 메커니즘입니다
키오스크 화면의 매핑	iptables 규칙(DNAT)	호스트 포트에 온 요청의 도착지를 컨테이너 포트에 바꿔치기하는 한 줄짜리 변환 규칙입니다
식당 주방	컨테이너 포트	컨테이너 안에서 앱이 듣고 있는 포트(<code>:80</code> 등)입니다

`docker run -p 8080:80` 을 실행하면 Docker는 iptables에 "호스트의 8080 포트에 들어온 요청을 컨테이너의 80 포트에 전달"이라는 규칙을 한 줄 씌어둡니다. 이제 브라우저가 `localhost:8080` 을 호출하면 iptables가 이 규칙대로 요청의 도착지를 컨테이너 80 포트에 바꿔치기하고 docker0를 거쳐 해당 컨테이너로 전달합니다. 사용자는 이 변환을 모르고 "localhost에 접속했더니 컨테이너 안의 페이지가 나왔네" 정도로만 경험합니다.

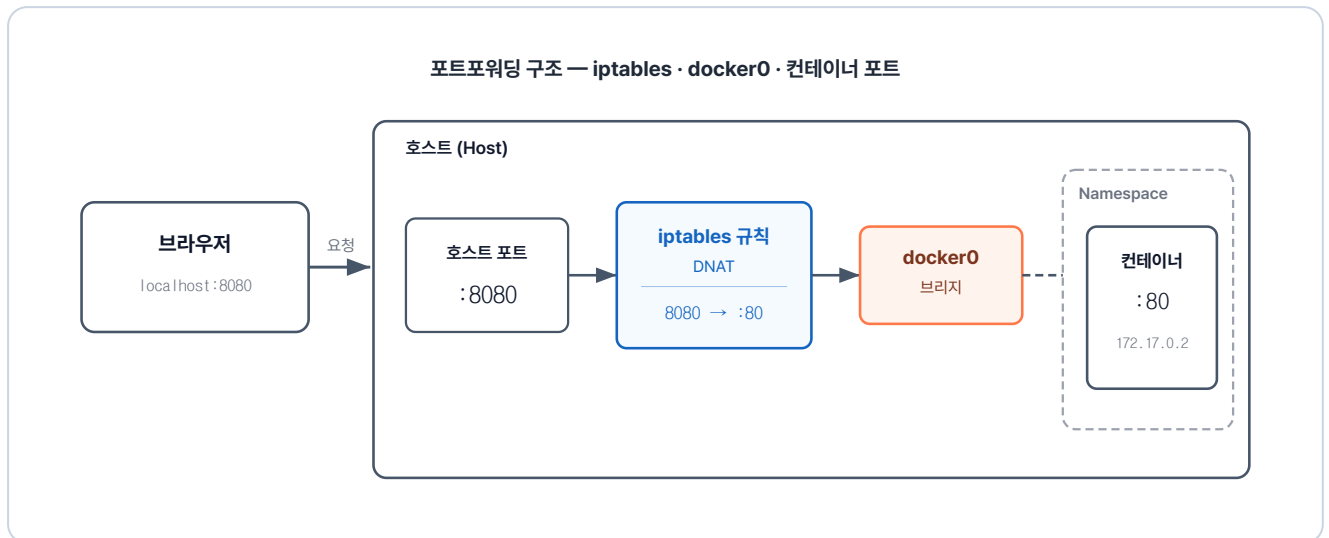


그림 2-15. 호스트 포트가 컨테이너 포트에 연결되는 구조

'외부에서 들어올 땐 결국 호스트 포트에 들어오는구나. iptables가 그 안에서 컨테이너 포트에 길을 바꿔주고, 컨테이너 IP는 외부에 노출할 필요가 없네.'

오픈이가 다음으로 알아본 건 컨테이너끼리 IP가 아닌 이름으로 부르는 방법이었습니다.

2.5.4 이름으로 찾기 — 홀의 안내 지도

컨테이너끼리 통신하려면 결국 상대 IP를 알아야 합니다. 그런데 컨테이너가 둘, 셋 늘어나면 IP를 일일이 외우고 적어두는 게 영 번거롭습니다.

'인터넷은 naver.com 같은 도메인 주소만 알면 알아서 연결되잖아. 컨테이너도 이름으로 찾을 수는 없나.'

푸드코트로 옮겨 보면, 손님은 식당 이름은 알지만 푸드코드 내에서 식당이 어디 있는지는 알 수 없습니다. 그럴 때 홀에 걸린 **안내 지도**에 "**B식당 = 2구역**"이라고 표시되어 있으면, 이름을 보고 위치를 쉽게 찾을 수 있습니다.

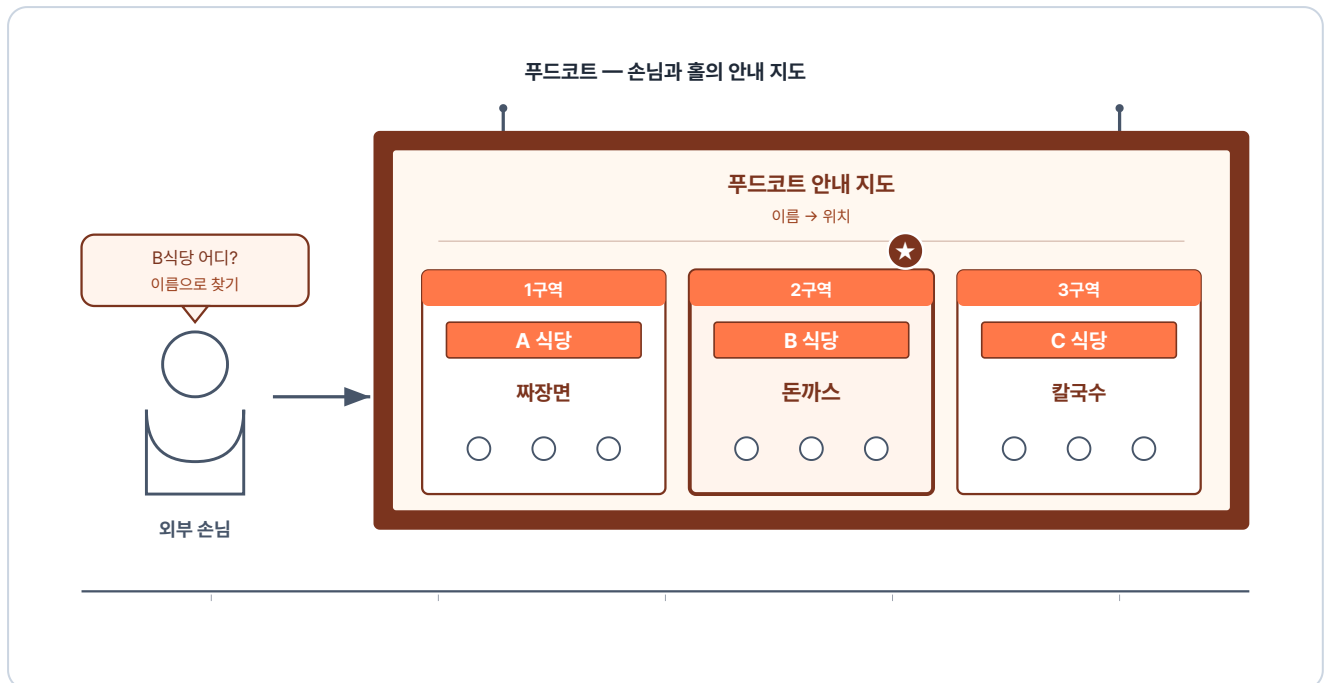


그림 2-16. 안내 지도가 이름을 위치로 바꿔주는 비유

이 비유를 IT로 옮기면 두 가지가 짝지어집니다.

푸드코트	IT 용어	한 줄 설명
안내 지도	Docker DNS	컨테이너 이름을 IP로 자동 변환해주는 내장 DNS(Domain Name System) 서버입니다.
같은 푸드코트 안	사용자 정의 네트워크	같은 네트워크에 묶인 컨테이너끼리만 이름으로 통신할 수 있습니다.

새 컨테이너가 뜰 때마다 이름과 IP가 자동으로 Docker DNS에 등록됩니다. 그 다음 `app` 컨테이너가 `db` 라는 이름으로 요청을 보내면 Docker DNS가 먼저 받아 `db` 를 `172.17.0.3` 으로 변환해 알려주고 app은 그 IP로 다시 요청을 보냅니다. 컨테이너가 다시 떠서 IP가 바뀌어도 이름은 그대로라 코드를 수정할 필요가 없습니다.

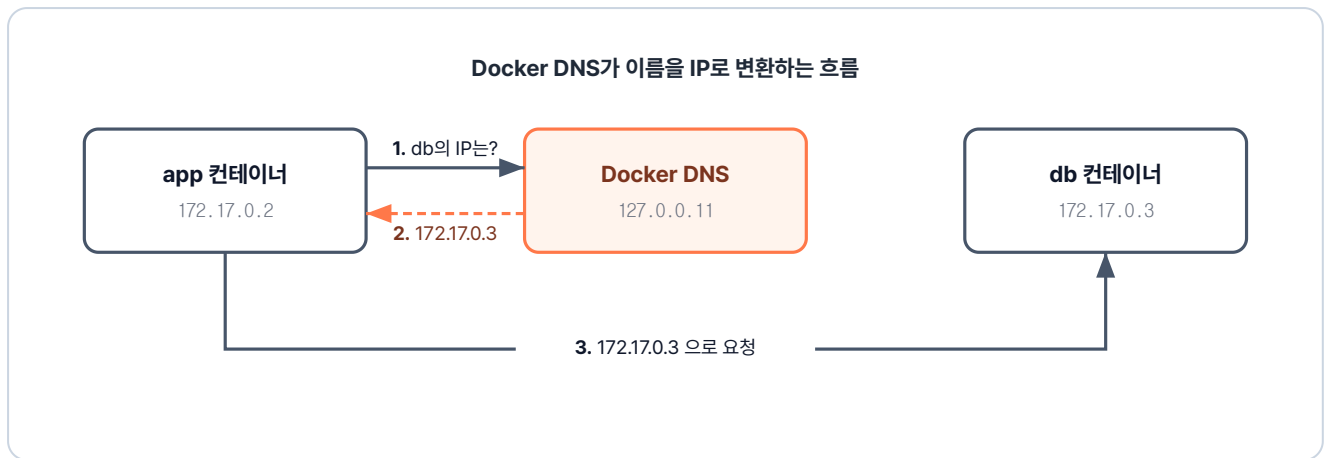


그림 2-17. Docker DNS가 이름을 IP로 변환

다만 이 자동 DNS는 **사용자 정의 네트워크(User-defined Network)** 안에서만 켜집니다. 지금까지 본 기본 네트워크(docker0)에서는 작동하지 않으므로, 사용자가 직접 네트워크를 만들어 컨테이너를 묶어둬야 이름으로 통신할 수 있습니다. 이 차이는 다음 챕터에서 직접 만들어보며 확인합니다.

'naver.com처럼 컨테이너도 이름으로 부르면 되는구나. IP는 일일이 외울 필요 없겠네.'

컨테이너 통신까지 파악한 오픈이는 이제 그 내부가 궁금해졌습니다. 컨테이너 안은 작은 리눅스 환경과 같아서, 내부를 살펴보려면 가장 먼저 기본 리눅스 명령어부터 알아보기로 했습니다.

2.6 컨테이너 내부 — 격리된 작은 리눅스

리눅스를 연습하려면 리눅스 환경이 필요합니다. **Ubuntu**는 가장 널리 쓰이는 배포판 중 하나라 자료가 풍부하고, Docker Hub에 공식 이미지가 올라와 있어서 바로 받아 띄울 수 있습니다. 오픈이는 포트포워딩을 사용해 ubuntu 컨테이너 내부로 들어갔습니다.

```
# -it: 컨테이너 안 터미널로 들어가기, -p 80:80: 포트포워딩
docker run -it -p 80:80 ubuntu
```

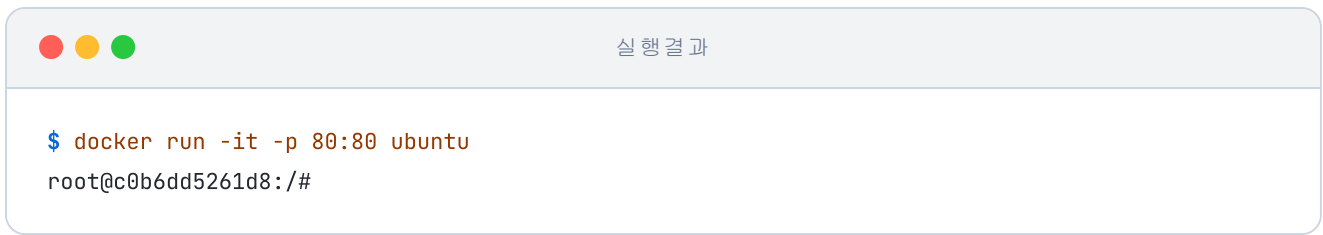


그림 2-18. Ubuntu 컨테이너 안으로 들어간 상태

프롬프트가 `root@...#` 모양으로 바뀌었습니다. 방금까지 호스트의 터미널이었는데 이제는 컨테이너 안의 셸입니다. 호스트와는 분리된 공간에 들어와 있다는 감각이 옵니다.

셸(Shell): 사용자와 커널 사이에서 명령을 주고받는 프로그램입니다. 사용자가 셸에 명령을 내리면 셸이 커널에 그 명령을 전달하고 커널이 프로세스를 만들어 실제 작업을 수행합니다.

2.6.1 자주 쓰는 리눅스 명령어

안으로 들어왔지만 당장 뭘 할 수 있는지가 막막했습니다. 명령어를 모르면 실습에 들어가도 매번 검색해야 흐름이 끊깁니다. 오픈이는 본격적으로 실습해보기 전에, 앞으로 자주 쓸 리눅스 명령어부터 한 표로 정리해봤습니다.

용도	명령	설명
위치 확인	<code>pwd</code>	현재 위치
이동	<code>cd <경로></code>	폴더 이동
목록	<code>ls</code> , <code>ls -la</code>	파일/폴더 목록 (숨김 포함)
폴더 생성	<code>mkdir <이름></code>	폴더 만들기
파일 생성	<code>touch <이름></code>	빈 파일
복사	<code>cp <원본> <사본></code>	파일 복사
이동/이름변경	<code>mv <원본> <대상></code>	이동 또는 rename
삭제	<code>rm <파일></code> , <code>rm -r <폴더></code>	삭제
패키지 설치	<code>apt update && apt install -y <이름></code>	패키지 설치
프로세스	<code>ps -ef</code> , <code>kill <PID></code>	실행 중 프로세스 / 종료
검색	<code>find <경로> -name <이름></code>	파일 검색
로그	<code>tail -n <수> <파일></code>	마지막 N줄

경로를 쓸 때는 **절대 경로**와 **상대 경로**를 구분해야 합니다.

절대 경로 / 상대 경로

둘의 차이는 **어디서부터 시작해서 읽느냐**에 있습니다.

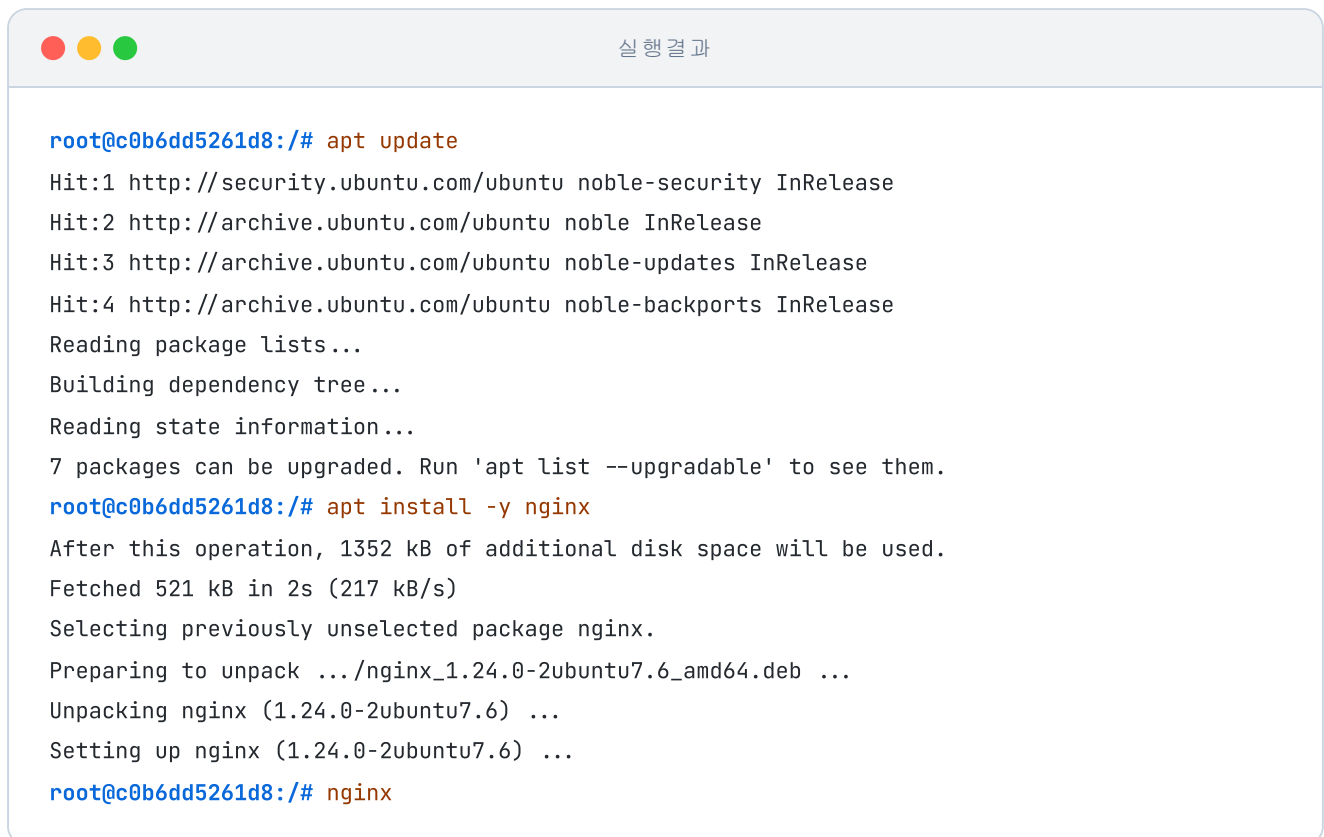
- **절대 경로** : **루트 경로(/)** 부터 시작해서 목적지까지의 전체 주소를 명시합니다. 현재 위치가 어디든 같은 곳을 가리킵니다. 리눅스에서는 결과적으로 `/` 로 시작하는 형태가 됩니다. 예: `/bin`, `/home/ubuntu/docs`.
- **상대 경로** : **현재 디렉터리**를 기준으로 목적지까지의 경로를 나타냅니다. 현재 위치에 따라 같은 표기가 다른 곳을 가리킬 수 있습니다. 예: `bin` (현재 디렉터리 아래 bin), `../etc` (상위 디렉터리의 etc).

2.6.2 실습: 컨테이너 안에 nginx 깔아보기

기본 명령어를 손에 익힌 오픈이는, 앞서 도커 이미지로 받아서 실행했던 nginx를 직접 설치해 보기로 했습니다. 이미지로 받을 땐 한 줄로 끝났던 과정을 손으로 밟아보면, 앞서 배운 이미지가 무엇을 담고 있었는지 역으로 보입니다.

컨테이너 안은 최소 구성이라 필요한 패키지는 직접 설치해야 합니다. 오픈이는 패키지 목록을 갱신한 뒤 nginx를 설치하고 실행했습니다.

```
apt update          # 최신 패키지 목록으로 업데이트
apt install -y nginx # nginx 설치
nginx               # nginx 실행
```



A terminal window titled "실행 결과" (Execution Result) showing the output of the commands. The terminal has a light blue header bar with three colored window control buttons (red, yellow, green) on the left. The output text is as follows:

```
root@c0b6dd5261d8:/# apt update
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists...
Building dependency tree...
Reading state information...
7 packages can be upgraded. Run 'apt list --upgradable' to see them.
root@c0b6dd5261d8:/# apt install -y nginx
After this operation, 1352 kB of additional disk space will be used.
Fetched 521 kB in 2s (217 kB/s)
Selecting previously unselected package nginx.
Preparing to unpack .../nginx_1.24.0-2ubuntu7.6_amd64.deb ...
Unpacking nginx (1.24.0-2ubuntu7.6) ...
Setting up nginx (1.24.0-2ubuntu7.6) ...
root@c0b6dd5261d8:/# nginx
```

그림 2-19. nginx 설치와 실행 결과

설치 로그가 마무리되며 Nginx 설치가 끝났습니다. 오픈이는 설정이 제대로 반영되었는지 확인하기 위해 현재 활성화된 포트를 점검해 보기로 했습니다. `net-tools` 패키지를 추가로 설치한 뒤, 명령어를 입력해 Nginx가 네트워크상에서 정상적으로 대기 중인지 확인했습니다.

```
apt install -y net-tools # net-tools: 네트워크 확인 도구 모음
netstat -nlpt           # 열려 있는 TCP 포트 확인
```



실행 결과

```
root@c0b6dd5261d8:/# apt install -y net-tools
Need to get 204 kB of archives.
After this operation, 811 kB of additional disk space will be used.
Fetched 204 kB in 2s (104 kB/s)
Selecting previously unselected package net-tools.
Preparing to unpack .../net-tools_2.10-0.1ubuntu4.4_amd64.deb ...
Unpacking net-tools (2.10-0.1ubuntu4.4) ...
Setting up net-tools (2.10-0.1ubuntu4.4) ...

root@c0b6dd5261d8:/# netstat -nlpt
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address Foreign Address State PID/Program name
tcp 0 0 0.0.0.0:80 0.0.0.0:* LISTEN 348/nginx: master p
tcp6 0 0 :::80 :::* LISTEN 348/nginx: master p
```

그림 2-20. 포트 상태 확인

80 포트가 열려 있는 것을 확인했습니다. 다음은 브라우저로 접속할 차례입니다. 주소창에 `localhost:80` 을 쳤습니다.

localhost



Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

그림 2-21. nginx 환영 페이지 응답

익숙한 nginx 환영 페이지가 떴습니다. 컨테이너를 띄울 때 `-p 80:80` 으로 포트를 열어둔 덕에, 컨테이너 안의 nginx와 브라우저가 연결된 것입니다.

2.6.3 vim으로 파일 편집

서버 설치를 마쳤더라도 운영하다 보면 설정 파일을 수정해야 할 일이 생기기 마련입니다. 리눅스에서 파일을 편집하려면 전용 편집기가 필요한데, 오픈이는 **Vim**을 선택해 설치했습니다.

```
apt install -y vim # 터미널 편집기 vim 설치
```

설치 중 시간대 선택 화면

apt install vim 도중 Geographic area와 Time zone을 선택하는 화면이 나옵니다. 이때 **Asia, Seoul**을 각각 선택합니다. 이 설정은 컨테이너 안의 시간 표시와 로그 시각 기준이 되어, 한국 시간으로 동작하게 만들어 줍니다.

vim의 사용 명령어는 다음과 같습니다.

단계	동작	키
1	파일 열기/생성	<code>vim <파일명></code>
2	입력 모드	<code>i</code>
3	일반 모드로	<code>ESC</code>
4	저장 후 종료	<code>:wq</code>

오픈이는 감을 잡으려고 간단한 파일을 하나 만들어봤습니다.

```
vim test1.txt # test1.txt 파일 생성
```

`i` 를 눌러 입력 모드로 전환하고 내용을 작성한 뒤 `ESC` → `:wq` 로 저장합니다.



그림 2-22. vim 편집 화면

저장이 잘 됐는지 `cat` 으로 내용을 확인했습니다.

```
cat test1.txt # 파일 내용 출력
```

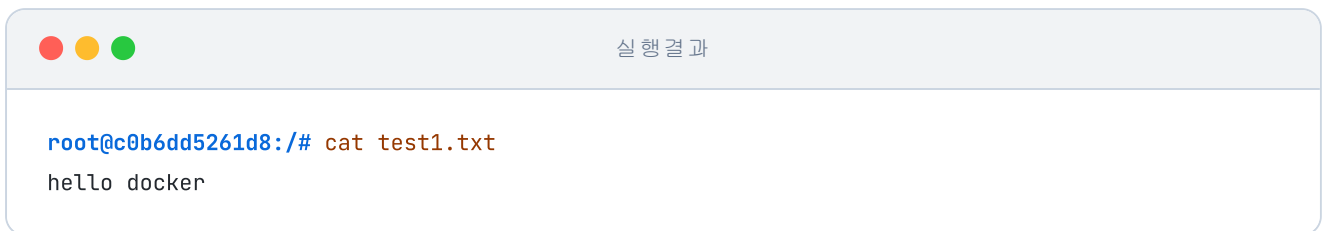


그림 2-23. 파일 내용 출력

방금 쓴 문장이 그대로 찍혀 나왔습니다.

그리고 보니 아까 깔았던 nginx의 기본 페이지는 어디에 놓여 있을까 궁금해졌습니다. 오픈이는 `find` 로 `index.html` 파일의 위치를 찾아봤습니다.

```
find / -name index.html # index.html 파일 위치 검색
```

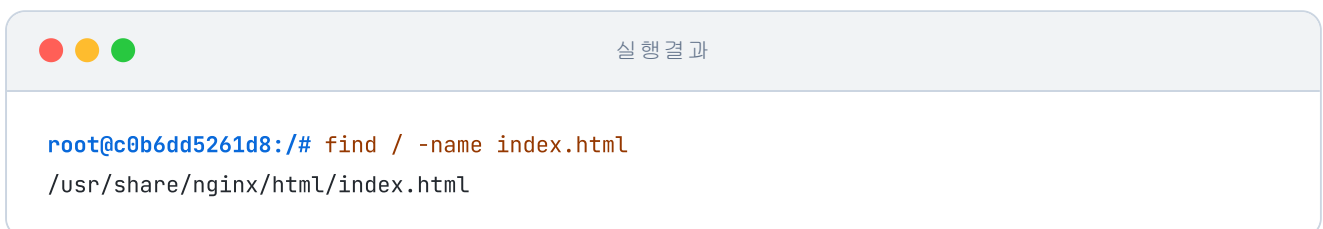


그림 2-24. 파일 검색 결과

컨테이너에서 빠져나올 때는 `exit` 입니다. 그런데 `exit` 를 치고 `docker ps` 를 쳐보니, 방금까지 돌던 ubuntu 컨테이너가 목록에서 사라져 있었습니다.

'어? 안에서 나왔을 뿐인데 왜 꺼지지!'

내부 구조는 익혔는데 **언제 살아 있고 언제 죽는지는** 아직 감이 없다는 뜻이었습니다. 컨테이너가 뜻밖에 꺼지면 곤란하니, 오픈이는 이 규칙부터 정리해보기로 했습니다.

2.7 포그라운드와 백그라운드

2.7.1 exit vs detach

오픈이는 같은 상황을 한 번 더 만들어 비교했습니다. 이름을 붙여 띄운 뒤 `exit` 로 나와봤습니다.

```
docker run -it --name dead ubuntu # 실행 후 exit로 나옴
exit
docker ps # 종료 여부 확인
```

실행 결과

```
root@f9a2b3c1d4e5:/# exit
exit
$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
```

그림 2-25. exit로 빠져나오면 컨테이너가 종료

예상대로 dead는 목록에 없었습니다.

오픈이는 `exit` 말고 다른 방식도 있는지 찾아봤습니다. `CTRL+P → CTRL+Q` 를 쓰면 컨테이너는 그대로 두고 쉘만 빠져나올 수 있습니다. 같은 방식으로 한 번 더 띄우고 이 조합으로 빠져나왔습니다.

```
docker run -it --name alive ubuntu # CTRL+P → CTRL+Q로 나옴
docker ps # 종료 여부 확인
```



```

root@17943f60b5cd:/#
$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
17943f60b5cd ubuntu "/bin/bash" Less than a second ago Up Less than a second alive
    
```

그림 2-26. CTRL+P → CTRL+Q로는 컨테이너가 유지

이번에는 alive가 살아 있었습니다. 같은 `docker run` 인데 나오는 방식에 따라 결과가 같았습니다.

방식	하는 일	결과
<code>exit</code>	컨테이너의 메인 프로세스(bash)를 종료	컨테이너가 종료
<code>CTRL+P → CTRL+Q</code>	터미널과 컨테이너의 연결만 끊음	컨테이너는 살아 있음

2.7.2 백그라운드 실행 조건

'그럼 처음부터 `-d`만 주면 어떨까.'

오픈이는 nginx 때 봤던 `-d` 옵션을 사용해 ubuntu 백그라운드로 실행해 보기로 했습니다.

```

docker run -d ubuntu # ubuntu를 백그라운드로 실행
docker ps           # 실행 중인 컨테이너 목록 확인
    
```

```

$ docker run -d ubuntu
0b74e66f5b0f3f4cb30b353503f2ef0c162e63f19be512e938f28e07d01dc1e7
$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
    
```

그림 2-27. ubuntu는 -d만으로는 바로 종료

'어 왜 실행 목록에 없지?'

실행 중인 프로세스 목록에 ubuntu가 없었습니다. 컨테이너가 실행되자마자 바로 종료되었기 때문입니다. 반면 이전에 실행했던 nginx는 똑같은 -d 옵션으로도 잘 돌아갔습니다.

오픈이는 그 원인을 찾아 **메인 프로세스**의 개념을 알게 되었습니다. 도커 컨테이너는 시작과 동시에 실행되는 메인 프로세스가 종료되면 컨테이너도 함께 종료됩니다. 이 관점에서 두 이미지를 비교해 보니 차이가 명확했습니다.

먼저 nginx는 '**웹 서버로서 요청을 대기하는 것**' 이 목적입니다. 그래서 메인 프로세스인 **nginx**는 요청이 올 때까지 계속 깨어 있는 데몬으로 동작합니다. 백그라운드에서도 '대기'라는 할 일이 명확히 있기 때문에 프로세스가 죽지 않고 컨테이너를 계속 살려두는 것입니다.

데몬(Daemon) : 백그라운드에서 요청을 기다리며 계속 실행되는 프로그램입니다. nginx나 데이터베이스처럼 '서버' 역할을 수행하며, 요청을 기다리는 것 자체가 일이라 터미널이 없어도 종료되지 않습니다.

반면 ubuntu는 '**사용자에게 명령어를 입력받는 것**' 이 목적입니다. 그래서 메인 프로세스인 **bash**는 사용자의 입력을 받는 터미널이 있어야 합니다. **-d** 옵션을 줘서 터미널이 연결되지 않으면, bash는 더 이상 할 일이 없다고 판단해 종료됩니다.

bash(셸) : 사용자의 명령어를 운영체제에 전달하는 창구입니다. 입력받을 터미널이 없으면 할 일이 사라져 프로세스가 종료됩니다.

ubuntu를 백그라운드로 살리려면 **-dit** (**-d** 백그라운드, **-i** 사용자와 상호작용, **-t** 터미널 화면)을 쓰거나 메인 프로세스를 다른 걸로 바꿔야 합니다.

2.7.3 CMD로 메인 프로세스 바꾸기

이번에는 ubuntu의 메인 프로세스를 유지하기 위해 메인 프로세스를 바꾸는 테스트를 해보기로 했습니다. bash 대신 **sleep 1000** 을 메인으로 지정했습니다.

```
docker run -d ubuntu sleep 1000 # bash 대신 sleep을 메인으로
docker ps                       # 1000초 동안 살아 있음
```

실행 결과

```
$ docker run -d ubuntu sleep 1000
7aca4e791643d3a39917aecc97868a62f5c1f6fa3df1ac63248e703650835cd3
$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
7aca4e791643 ubuntu "sleep 1000" Less than a second ago Up Less than a second goofy_bose
462ed99bd1fa ubuntu "/bin/bash" 2 seconds ago Up 1 second reverent_lehmann
46ef77d616b4 nginx "/docker-entrypoint...." 5 seconds ago Up 4 seconds 80/tcp vil
```

그림 2-28. CMD를 sleep으로 바꾸면 유지

이번엔 ubuntu가 목록에 남아 있었습니다. sleep은 터미널이 없어도 1000초 동안 할 일이 있는 프로세스이기 때문입니다. 이렇게 기본 실행 명령을 덮어쓰는 것이 **CMD**입니다.

CMD: 컨테이너가 시작될 때 실행되는 **기본 프로세스**를 정의하는 설정입니다. `docker run <이미지> <CMD옵션>` 형태로 직접 주면 이미지의 메인 프로세스 대신 CMD에 넣은 설정으로 덮어씁니다.

2.7.4 attach vs exec

오픈이는 백그라운드로 띄운 컨테이너에 다시 들어가는 방법이 궁금해졌습니다. 알아보니 **메인 프로세스에 직접 연결하는 `attach`**, **새 프로세스를 하나 더 띄우는 `exec`** 두 가지 방법을 확인했습니다. 결과는 비슷해 보이지만 내부에서 일어나는 일이 다르다고 해서, 두 가지를 직접 비교해봤습니다.

그림처럼 `attach`는 컨테이너 안에서 이미 돌고 있는 메인 프로세스를 연결합니다. 같은 프로세스를 공유하기 때문에 여기서 잘못 빠져나와 프로세스를 종료하면 컨테이너 전체가 같이 꺼집니다.

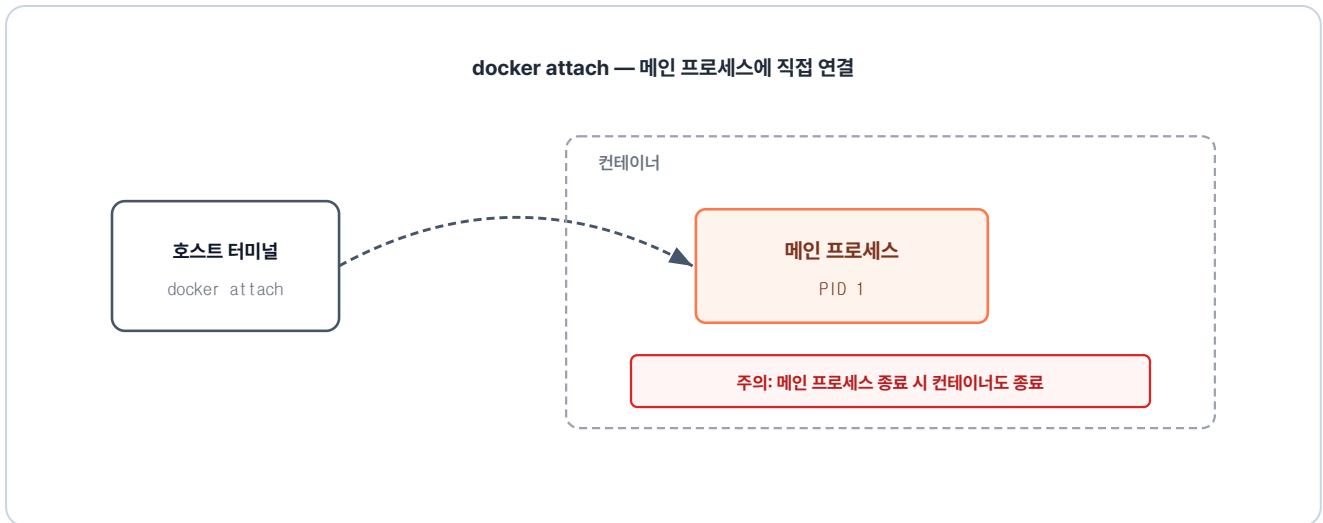


그림 2-29. attach는 이미 떠 있는 PID 1 프로세스에 터미널을 직접 꽂는 방식

오픈이는 먼저 ubuntu를 백그라운드로 띄우고, 그 컨테이너에 `docker attach` 로 접속했습니다.

```

docker run -dit ubuntu # ubuntu 백그라운드 실행
docker ps             # 실행중인 컨테이너 확인
docker attach d2b1    # attach로 접근
  
```

실행 결과

```

$ docker run -dit ubuntu
d2b18acc2cf05a40644f0ca8536cb50a0ff560a1be0c9ce2fbf5ddbbb a0e729c
$ docker ps
CONTAINER ID  IMAGE  COMMAND                  CREATED          STATUS          PORTS          NAMES
d2b18acc2cf0  ubuntu "/bin/bash"              Less than a second ago Up               Less than a second angry_engelbart
$ docker attach d2b1
root@d2b18acc2cf0:/#
  
```

그림 2-30. attach로 접근

ubuntu의 입력 터미널이 나타났습니다. `attach` 는 **메인 프로세스**에 직접 연결되므로, 여러 터미널에서 붙어도 모두 같은 화면을 공유합니다. 다만 이 상태에서 잘못 빠져나와 메인 프로세스를 종료하면 컨테이너 전체가 꺼집니다. 오픈이는 `CTRL+P → CTRL+Q` 로 안전하게 빠져나왔습니다.

이번에는 같은 컨테이너에 `exec` 로 다시 들어가봤습니다.

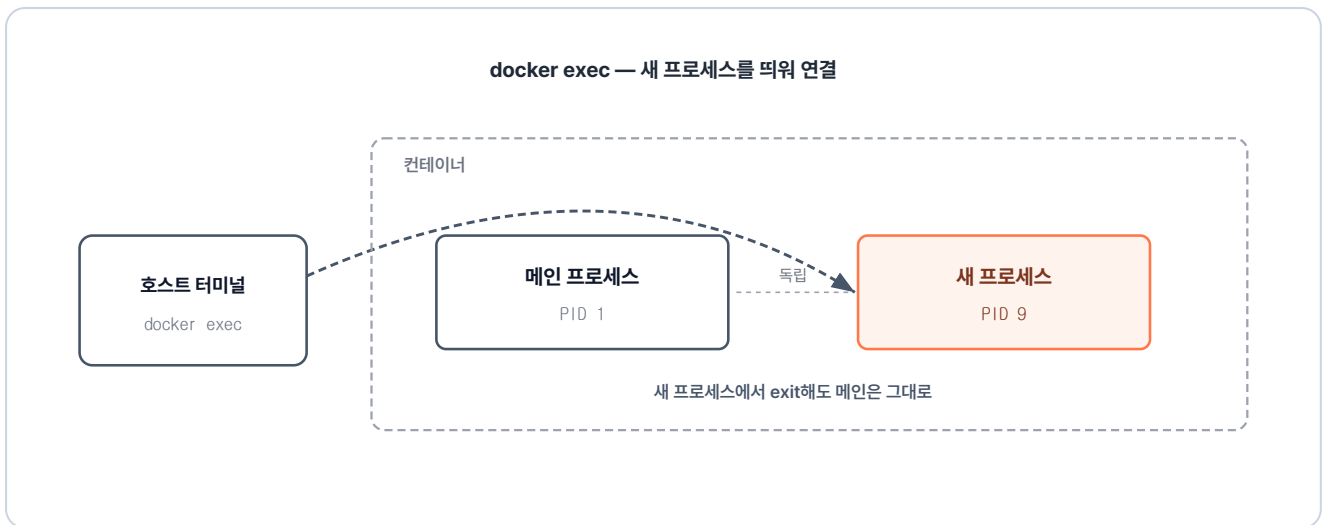


그림 2-31. exec는 컨테이너 안에 새 bash 프로세스를 띄우고 내 터미널에 연결하는 방식

그림처럼 `exec`는 메인 프로세스에 직접 연결하는 대신, 컨테이너 안에 **별도의 bash 프로세스를 새로 하나 띄우고** 거기에 내 터미널을 연결합니다. 메인 프로세스와 분리되어 있어 이쪽에서 `exit`로 나와도 메인 프로세스는 종료되지 않습니다.

```
docker exec -it d2b1 bash # 실행 중인 컨테이너에 새 bash 접속
```

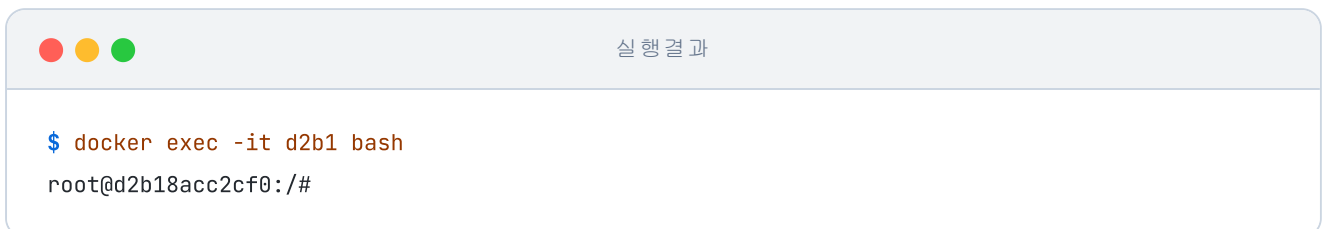


그림 2-32. exec로 접근

`exec`가 정말 새 프로세스를 만드는지 눈으로 확인하고 싶어서, 오픈이는 호스트에서 터미널을 하나 더 열고 컨테이너 내부의 프로세스 목록을 찍어봤습니다.

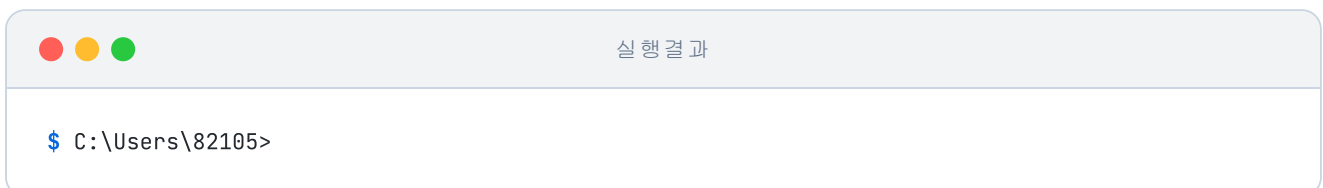


그림 2-33. 호스트에서 새 터미널 창 실행

```
docker exec d2b1 ps aux # 컨테이너 내부 프로세스 목록 확인
```

```

프로세스 목록 확인

$ docker exec d2b1 ps aux
USER PID %CPU %MEM VSZ RSS TTY STAT START TIME COMMAND
root 1 0.0 0.0 4588 3840 pts/0 S+ 03:29 0:00 /bin/bash
root 12 0.1 0.0 4588 3840 pts/1 S+ 03:34 0:00 bash
root 21 63.6 0.0 7888 4096 ? Rs 03:34 0:00 ps aux
    
```

그림 2-34. 프로세스 목록 확인

PID 1이 컨테이너의 메인 프로세스이고, **PID 12**가 방금 `exec`로 만들어진 프로세스입니다. 두 프로세스는 같은 컨테이너 안에서 환경을 공유하지만 터미널 세션은 서로 독립입니다. 그래서 `exec`에서 선언한 임시 변수가 메인 프로세스 세션에서는 보이지 않고, 반대도 마찬가지입니다.

방식	동작	위험성
<code>docker attach <ID></code>	메인 프로세스(PID 1)에 직접 연결	잘못 빠져나오면 메인도 종료되고 컨테이너도 꺼짐
<code>docker exec -it <ID> bash</code>	새 프로세스를 만들어 접근	안전. 빠져나와도 메인에 영향 없음

'attach는 메인, exec는 새 방. 앞으로는 exec만.'

여기까지 공부하니 컨테이너의 생명 주기는 더 이상 헛갈리지 않았습니다. 그런데 한 가지 생각이 오픈이를 붙잡았습니다. 앞서 Ubuntu 컨테이너에 패키지를 설치하고 파일을 추가했는데, 컨테이너를 내리는 순간 지금까지 했던 작업이 사라지는 문제였습니다.

'내가 수정한 이 상태를 기록해두려면 어떻게 해야 하지?'

필요한 건 지금 컨테이너 상태를 그대로 기록해 두는 방법입니다.

2.8 commit — 실행 중인 컨테이너를 이미지로

그 일을 하는 명령이 `docker commit`입니다. 이 명령어는 컨테이너의 현재 상태를 그대로 이미지로 저장합니다.

지금까지 오픈이는 남이 올려둔 이미지(ubuntu, nginx, tomcat)를 Docker Hub에서 받아 쓰기만 했습니다. 반대로, 직접 수정한 컨테이너를 이미지로 만들어 Docker Hub에 올릴 수도 있습니다. 그러면 다른 컴퓨터에서도 이 이미지를 받으면 같은 상태로 쓸 수 있습니다.

2.8.1 Tomcat으로 commit 실습하기

오픈이는 자기만의 이미지를 만들기 위해 Tomcat 이미지를 받아 컨테이너를 실행했습니다.

```
docker run -d -p 8080:8080 tomcat # 8080 포트로 Tomcat 백그라운드 실행
```

실행 후 브라우저에서 **localhost:8080** 으로 접속했더니 **404**가 떴습니다.



그림 2-35. Tomcat 404 에러 화면

'연결은 된 거 같은데 왜 404 에러가 뜨지?'

이유는 Docker Hub에서 내려받은 Tomcat 이미지에는 `webapps/ROOT/index.html` 파일이 없기 때문입니다.

오픈이는 이 경로에 index.html을 작성해보기로 했습니다.

```
docker exec -it 5fcd bash # 실행 중인 컨테이너 안으로 들어가기
cd /usr/local/tomcat/webapps # Tomcat의 웹앱 기본 경로로 이동
mkdir ROOT && cd ROOT # ROOT 폴더 생성 후 그 안으로 이동
apt update && apt install -y vim # 패키지 목록 갱신 후 vim 편집기 설치
vim index.html # index.html 파일을 만들고 간단한 HTML 작성 후 저장
```

vim 패키지로 index.html 생성 및 저장을 마치고 **localhost:8080** 로 다시 접속했더니 방금 만든 페이지가 응답되었습니다.

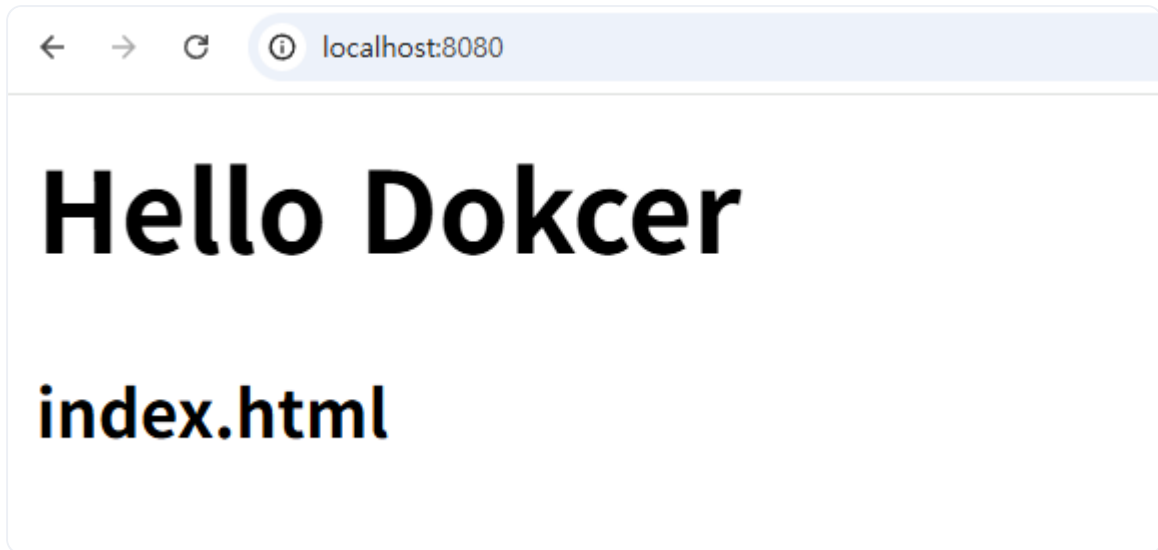


그림 2-36. index.html 응답 확인

이제 `docker commit <컨테이너ID> <본인-dockerhub-id>/<이미지이름>` 형식으로 명령어를 입력해 현재 상태를 기록합니다.

```
docker commit 5fcd coderyu5523/tomcat # 현재 컨테이너 상태를 새 이미지로 저장
```

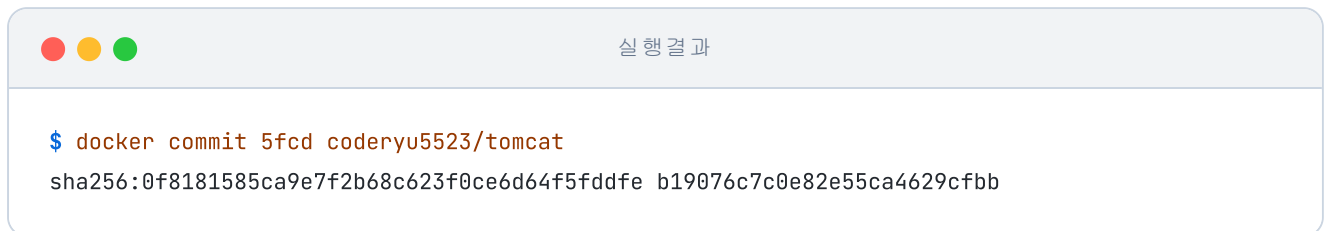


그림 2-37. 이미지 커밋 완료 화면

저장된 이미지가 목록에 잘 들어갔는지 확인했습니다.

```
docker images # 로컬에 저장된 이미지 목록 확인
```

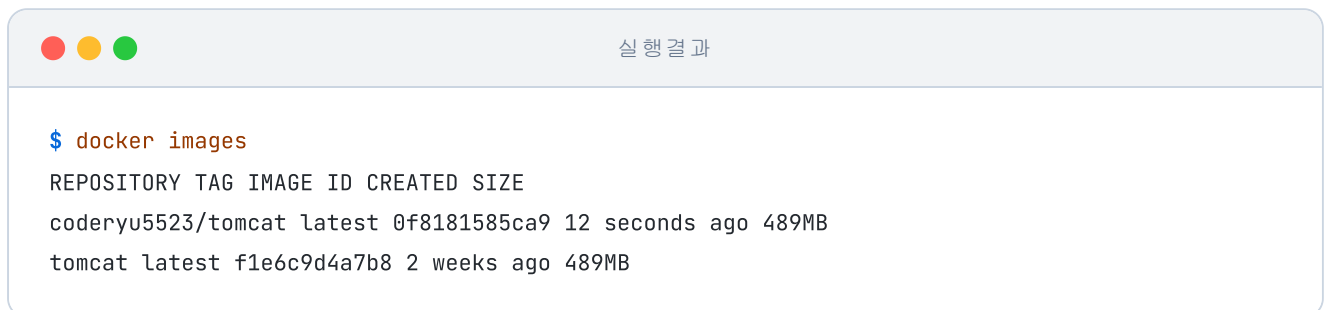


그림 2-38. 로컬 이미지 목록에 새 이미지 추가 확인

내가 만든 이미지를 목록에서 확인할 수 있습니다.

2.8.2 Docker Hub에 올리기

오픈이가 저장한 이미지는 로컬에만 있었습니다. 이 이미지를 Docker Hub에 올려두면 다른 PC나 서버에서 `docker pull`로 받아 쓸 수 있습니다.

```
docker login                                # Docker Hub 로그인
docker push <본인-dockerhub-id>/tomcat     # 로컬 이미지를 Hub로 업로드
```

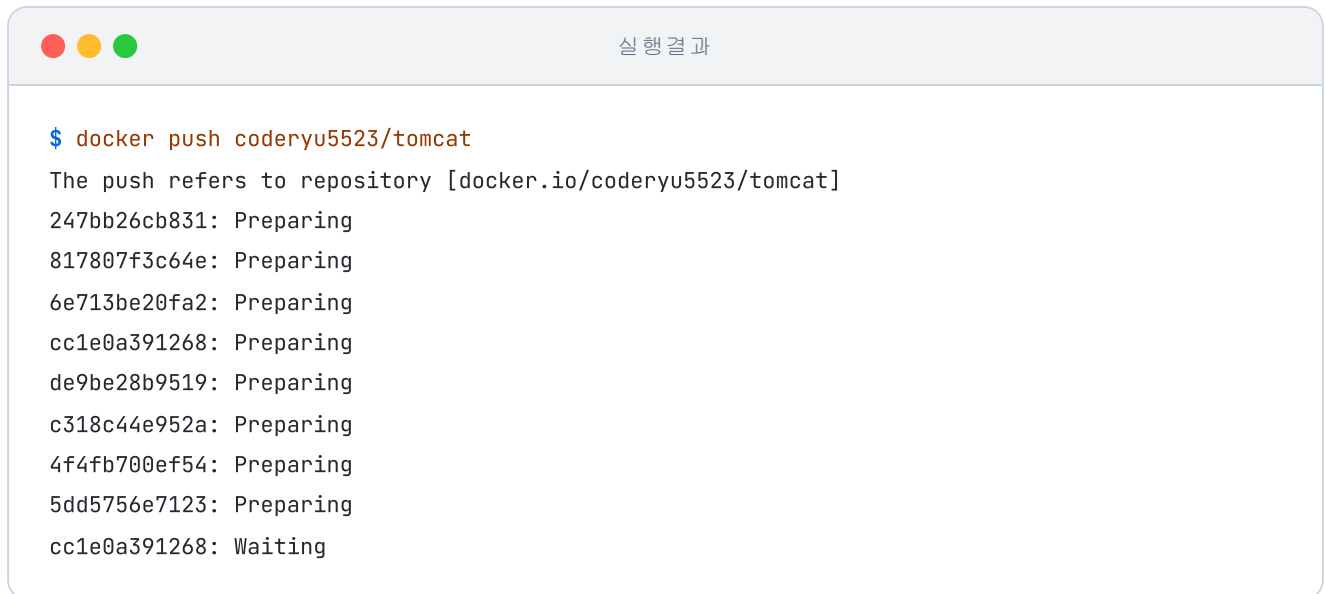


그림 2-39. docker push 실행 결과

업로드가 완료된 뒤 Docker Hub의 Repositories 탭을 열어보니 방금 올린 이미지를 확인할 수 있습니다.

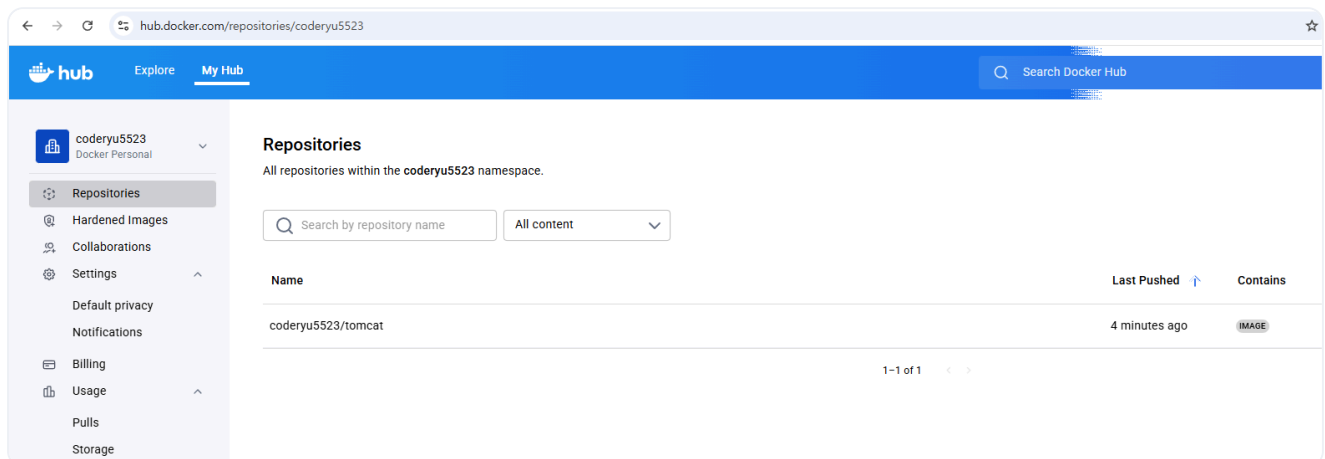


그림 2-40. Docker Hub 저장소 확인

2.9 마운트로 데이터 보존

Docker Hub에 이미지를 올리고 한숨 돌리려던 오픈이의 머릿속에 문득 한 가지 걱정이 스쳤습니다. 이미지에 담긴 것은 컨테이너의 초기 상태뿐이기 때문입니다.

컨테이너가 실행된 뒤 새로 쌓이는 데이터는 이미지에 저장되지 않습니다. 컨테이너가 삭제되는 순간 함께 사라져 버립니다. 데이터베이스처럼 실시간으로 데이터를 쌓는 컨테이너가 삭제되면, 그동안 저장한 데이터도 통째로 날아갑니다.

'컨테이너를 내려도 데이터는 남아 있어야 하는데.'

결국 데이터를 잃지 않으려면 컨테이너 외부에 보관해야 합니다. 이를 위해 컨테이너와 외부 저장소를 연결해주는 기능이 바로 **마운트**입니다.

Docker의 마운트는 두 가지 방식이 있습니다.

2.9.1 바인드 마운트: 호스트 폴더와 직접 연결

먼저 해본 방식은 호스트 PC의 특정 폴더를 컨테이너 내부의 경로와 연결하는 **바인드 마운트**입니다. USB를 컴퓨터에 꽂으면 그 안의 파일을 그대로 볼 수 있듯, 컨테이너도 호스트 PC의 폴더를 꽂아서 쓰는 방식입니다.

한쪽에서 파일을 수정하면 다른 쪽에서도 즉시 확인됩니다. USB를 뽑아도 안에 든 파일이 사라지지 않듯, 컨테이너를 삭제해도 파일은 호스트 PC에 그대로 남습니다.

바인드 마운트(Bind Mount) : 호스트 PC의 특정 폴더를 컨테이너 안 경로와 직접 연결하는 마운트 방식입니다. 사용자가 지정한 폴더를 컨테이너가 그대로 빌려 쓰기 때문에, 한쪽의 변경 사항이 실시간으로 양쪽에 반영됩니다. 어느 폴더를 쓸지는 사용자가 직접 지정합니다.

```
docker run -it --mount type=bind,src=<호스트 경로>,dst=<컨테이너 경로> <이미지>
```

> 형태로 명령어를 작성합니다.

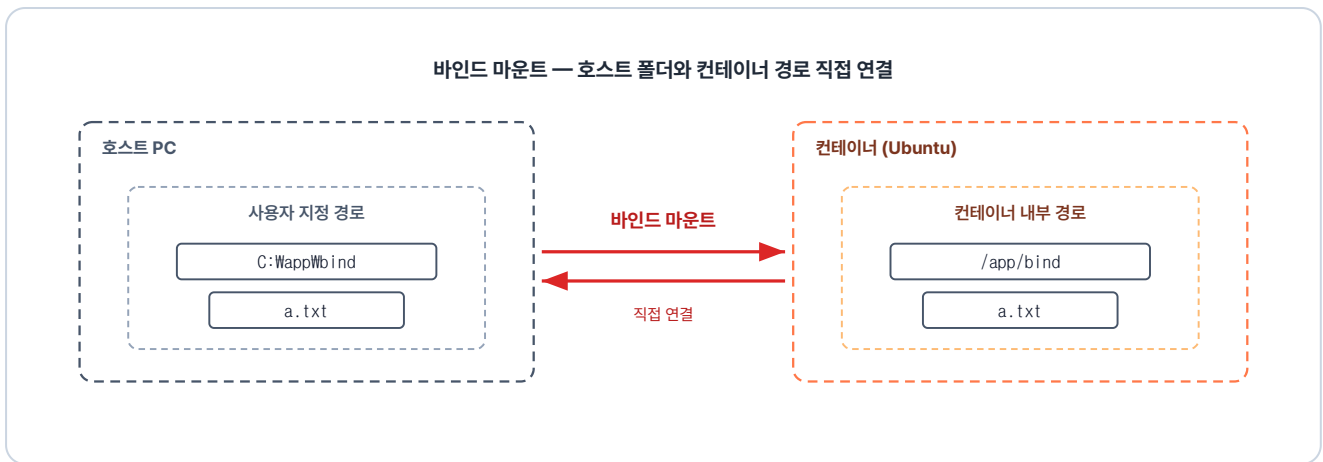


그림 2-41. 호스트 폴더와 컨테이너 경로가 같은 데이터를 바라보도록 묶인 상태

오픈이는 호스트의 `/c/app/bind` 경로에 폴더를 하나 만들고, 컨테이너 안의 경로와 직접 연결했습니다.

```
# 호스트 PC에 공유할 폴더 생성
mkdir -p ~/app/bind
# 호스트 폴더(src)와 컨테이너 경로(dst)를 연결
docker run -it --mount type=bind,src=/c/app/bind,dst=/app/bind ubuntu
```

컨테이너 안에서 `a.txt` 를 만들고, 호스트 쪽 폴더를 확인해 봤습니다.

```
touch /app/bind/a.txt # 컨테이너 안에서 파일 생성
ls C:/app/bind        # 호스트 터미널에서 확인
```

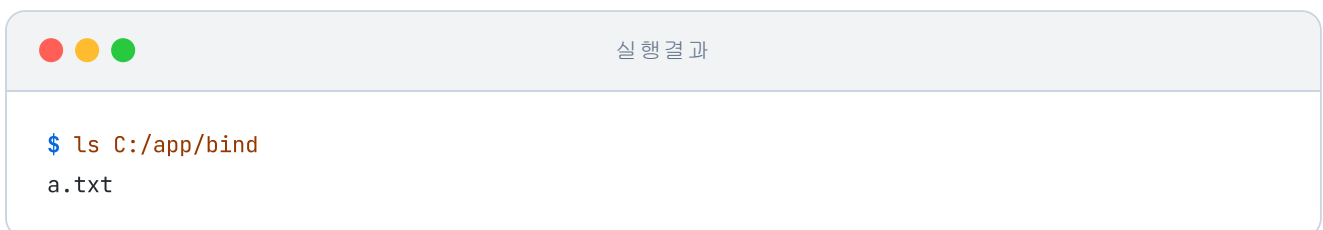


그림 2-42. 호스트 PC에서 같은 파일이 보이는 것을 확인

같은 파일이 양쪽에 있었습니다. 호스트에서 소스 코드를 고치면 컨테이너에 바로 반영되는 구조라, 개발할 때 쓰기 딱 맞는 방식입니다.

2.9.2 볼륨 마운트: Docker가 관리하는 저장소

바인드 마운트를 써보니 신경 쓰이는 점이 하나 있었습니다. 호스트의 어느 폴더를 쓸지, 경로를 어떻게 정리해둘지 사용자가 직접 관리해야 한다는 점입니다. 이 일을 Docker에 맡길 수는 없을까.

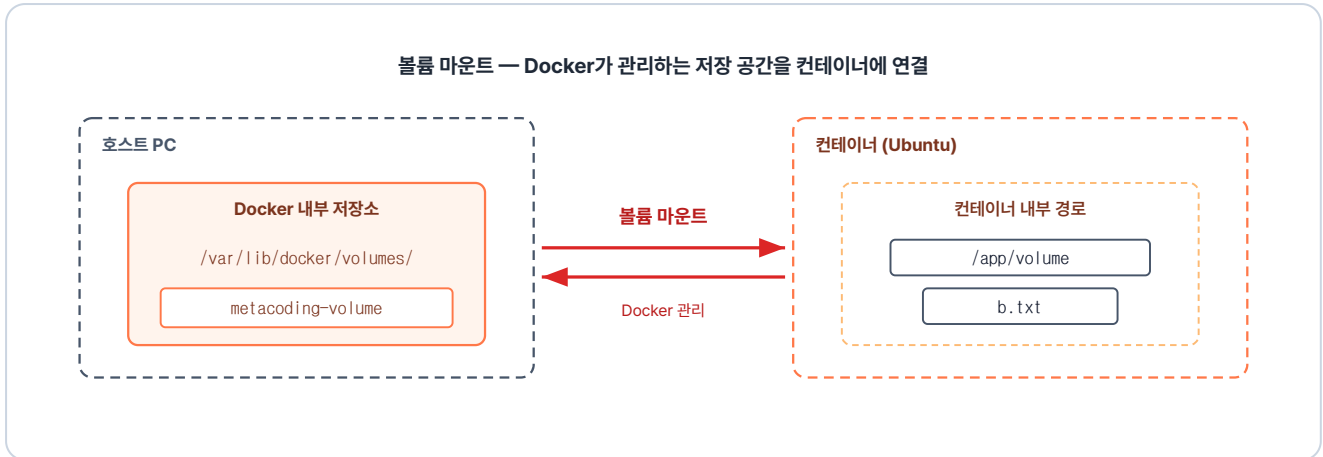


그림 2-43. Docker 엔진이 관리하는 내부 저장 공간에 데이터가 저장되는 구조

볼륨 마운트가 그 역할을 합니다. Docker 엔진이 자기 안에 별도 저장 공간을 두고, 사용자는 볼륨에 이름만 붙여 컨테이너와 연결합니다. 실제 저장 위치는 Docker가 맡아서 관리합니다.

볼륨 마운트(Volume Mount) : Docker 엔진이 자체적으로 관리하는 저장 공간(볼륨)을 컨테이너에 연결하는 마운트 방식입니다. 사용자는 볼륨 이름만 정하면 되고, 실제 저장 위치는 Docker가 알아서 관리합니다. 컨테이너가 사라져도 볼륨에 담긴 데이터는 그대로 남아 있어, 다른 컨테이너에서 같은 이름으로 다시 연결할 수 있습니다.

```
docker run -it --mount type=volume,src=<볼륨 이름>,dst=<컨테이너 경로> <
이미지> 형태로 명령어를 작성합니다.
```

오픈이는 `metacoding-volume` 이라는 이름으로 볼륨을 하나 만들면서 ubuntu 컨테이너를 띄웠습니다. 존재하지 않는 볼륨 이름을 주면 Docker가 자동으로 만들어 줍니다.

```
# 볼륨 마운트로 ubuntu 실행
docker run -it --mount type=volume,src=metacoding-volume,dst=/app/volume ubuntu
```



그림 2-44. 볼륨 마운트로 ubuntu 실행

컨테이너 안에 들어가자 `/app/volume` 폴더가 자동으로 만들어져 있었습니다. 오픈이는 이 폴더에 빈 파일을 하나 만들고, 컨테이너를 빠져나와 볼륨이 그대로 남아 있는지 확인했습니다.

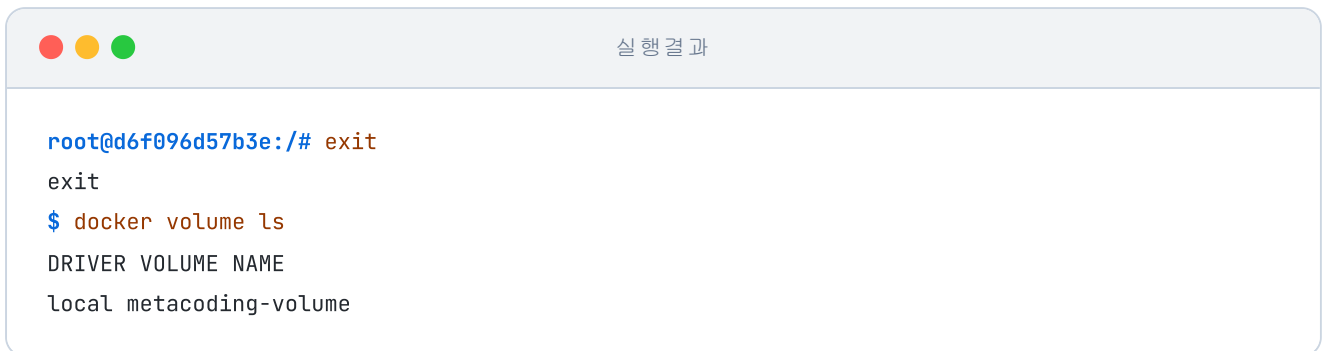
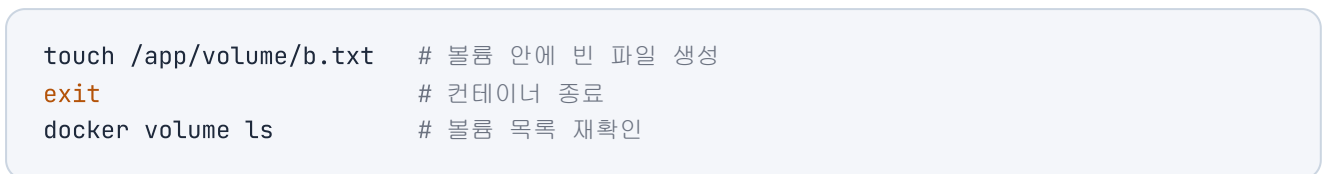


그림 2-45. 컨테이너가 사라져도 볼륨은 유지

컨테이너는 사라졌지만 볼륨은 목록에 그대로 남아 있었습니다. 이어서 같은 볼륨을 연결해 새 컨테이너를 다시 띄웠습니다.

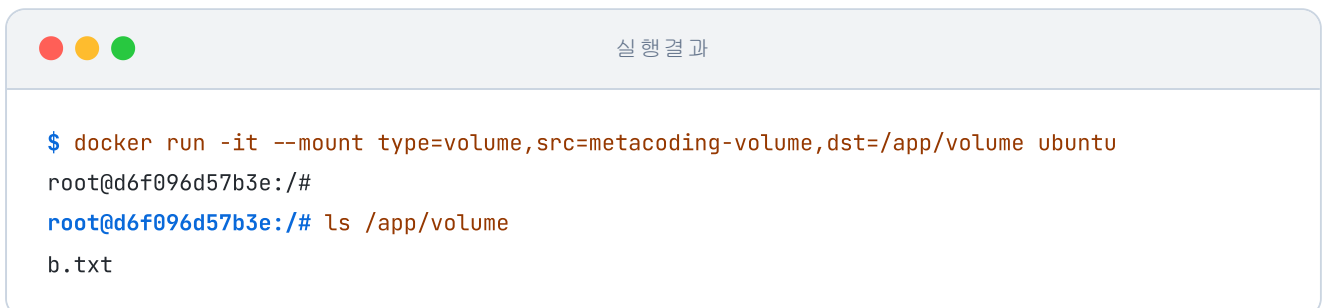


그림 2-46. 볼륨 데이터 재사용

방금 전에 만들었던 `b.txt`가 새 컨테이너 안에서도 그대로 보였습니다. 컨테이너를 내려도 데이터는 사라지지 않았습니다. 처음 던졌던 걱정이 풀리는 순간이었습니다.

이것만은 기억하자

- **컨테이너는 격리된 프로세스입니다.** 파일시스템·네트워크·프로세스 공간만 따로 쪼개고, 커널은 호스트와 공유합니다. VM보다 가볍고 빨리 뜹니다.
- **이미지는 설계도, 컨테이너는 찍혀 나온 결과입니다.** 붕어빵 틀 하나로 여러 붕어빵을 찍듯, 이미지 하나로 여러 컨테이너를 찍어냅니다.
- **컨테이너 통신은 docker0가 중심입니다.** 컨테이너끼리는 docker0로 묶이고, 외부 요청은 포트포워딩으로 들어옵니다.
- **메인 프로세스가 살아 있어야 컨테이너가 살아 있습니다.** 이미지의 CMD에 앱 실행 명령을 지정해두는 이유가 여기에 있습니다.
- **마운트는 컨테이너와 외부 저장소를 잇습니다.** 바인드는 호스트의 특정 폴더를, 볼륨은 Docker 엔진이 관리하는 저장소를 컨테이너에 연결합니다.

오픈이가 Docker라는 이름을 처음 들은 건 어제 낮이었습니다. 그 사이에 컨테이너로 앱을 돌리고, 이미지로 같은 환경을 어디서든 다시 띄울 줄 알게 됐습니다.

노트를 훑어보고 노트북 뚜껑을 닫았습니다. 책상 위엔 식은 커피 잔이 남아 있었습니다.

하지만 앞으로 해결해야 할 과제도 남아 있습니다. 매번 컨테이너에 접속해 수동으로 설정을 변경하고 commit 명령어로 이미지를 기록하는 방식은 비효율적입니다. 나아가 여러 개의 컨테이너를 동시에 실행하고 관리해야 하는 상황도 곧 마주하게 될 것입니다.